Subchat Trees: A Scalable Hierarchical Architecture for Multi-Threaded Dialogue and Context Isolation in LLMs

Mehedi Hasan Moon[1*], Kabbo Sarker Surjo[1] and Md. Akhtaruzzaman[1]

[1*]Department of Computer Science and Engineering, Military Institute of Science and Technology (MIST), Dhaka, Bangladesh.

*Corresponding author(s). E-mail(s): the.mehedi.hasan.moon@gmail.com;
Contributing authors: kabbosarker29@gmail.com;
akhter900@cse.mist.ac.bd;

Abstract

Current LLM chat interfaces are still organized mainly as linear transcripts: every topic, correction, side question, and follow-up is appended to the same chronological history. This design works for short single-topic exchanges, but it becomes fragile in realistic long conversations where users switch between tasks, return to earlier topics, or open nested follow-ups. Unrelated context can leak across topics, older details can be evicted or buried, and increasing the model's context length alone does not decide which part of the conversation should be active. We introduce *Subchat Trees*, a user-facing nested branching architecture for conversation management. Instead of treating the chat as one flat buffer, Subchat Trees represents dialogue as a tree of parent and child subchats, where each node maintains its own local buffer, rolling summary, and vector-indexed long-term memory. A user can create a focused subchat from a selected parent fragment, continue a detailed side discussion in that branch, and later return to the parent without contaminating sibling or ancestor contexts. This external conversation structure is model-agnostic and can improve LLM behavior without fine-tuning, because the model receives a cleaner branch-local context at inference time. We evaluate the approach on a 309-turn Variable-Block Randomized Interleaving (VBRI) benchmark derived from five LMSYS-Chat-1M conversations, covering 59 topic labels and 43 topic-recall probes. Across Qwen2.5 7B, 14B, and 32B AWQ models and buffer sizes $\boldsymbol{C} \in \{\mathbf{5}, \mathbf{10}, \mathbf{20}, \mathbf{40}\}$, Subchat Trees improves context-isolation F1 over the linear baseline in every model-buffer setting. The best absolute result occurs with the 32B model at $\boldsymbol{C} = \mathbf{20}$, where F1 improves

from 69.6% to 85.2% and topic pollution falls from 32.7% to 16.5%. The 14B model provides the strongest practical tradeoff, reaching 83.4% F1 at $\boldsymbol{C = 10}$ and 83.1% at $\boldsymbol{C = 20}$. Recall probes also show stronger topic retention: average ROUGE-1 improves over baseline in all tested settings, including 0.2476 to 0.4187 for 14B at $\boldsymbol{C = 20}$ and 0.3338 to 0.4283 for 32B at $\boldsymbol{C = 20}$. Although raw input tokens sometimes increase because cleaner context elicits fuller answers, tokens per correct answer decreases across all tested configurations. These results suggest that complex conversations need explicit external structure, not only longer context windows, and that nested branching is a practical interface-level method for improving long-running LLM reliability.

Keywords: Large Language Models, Conversational AI, Hierarchical Dialogue, Context Isolation, Retrieval-Augmented Generation, Multi-threaded Conversations

1 Introduction

Large language models now serve as the backbone of chat systems used for programming assistance, education, planning, research, and open-domain dialogue. Despite these gains, failures remain common in long, multi-topic conversations: earlier constraints are forgotten, separate tasks become mixed, and outdated turns keep influencing later responses. These pathologies persist because most chat interfaces still organize every exchange as a single chronological transcript, even when the underlying interaction contains multiple partially overlapping threads. As sessions grow, this flat structure increases context pollution and makes it harder for the model to recover the right information at the right time [1, 2].

At the same time, several research directions suggest that explicit structure can improve language-model behaviour. Chain-of-Thought, Self-Consistency, Tree-of-Thought, and Graph-of-Thought show that structured intermediate reasoning improves problem solving on complex tasks [3–6]. Conversation disentanglement studies show that mixed message streams naturally contain distinct threads, but recovering those threads after the fact is difficult and error-prone [7–9]. Hierarchical dialogue models, memory systems, retrieval augmentation, and role-based interaction frameworks likewise add structure at different levels of the stack [10–14].

More recent work treats multi-turn interaction as a distinct capability regime rather than a simple repetition of single-turn prompting. Long-context studies show that relevant evidence becomes harder to use when it is buried in the middle of an input sequence [1]. Dialogue-focused benchmarks and memory studies report similar degradation across long-running interactions, including failures in factual consistency, event recall, and persona tracking [2, 15–17]. Prompt compression can reduce token cost and improve information density, but it still operates over a fundamentally linear interaction record [18].

Consider a user who starts with Python debugging, shifts to python snakes for a biology question, opens a side discussion about Linux audio, and later returns to the original code. In a linear transcript, the latest query is interpreted against an interleaved history where keywords, pronouns, and instructions now refer to multiple

competing contexts. The result can be semantic drift, role confusion, privacy leakage between topics, and unnecessary token growth from carrying a broad history that no longer belongs to the current task.

The central issue, then, is not only how to store more context, but how to organize context so that unrelated conversational threads stop competing with one another. *Subchat Trees* addresses this problem at the interaction layer. Rather than asking the model to untangle a mixed transcript internally, the interface preserves topic structure explicitly through branch creation, local buffers, rolling summaries, and branch-scoped retrieval. The goal is to keep active context focused while still allowing long-term information to remain accessible.

1.1 Overview of Subchat Trees

Subchat Trees organizes a session as a tree of related dialogue nodes rather than a single transcript. The design goal is to keep each active branch semantically focused while still allowing information to persist through summarization and retrieval. The architecture centers on four design elements:

- Each conversation node maintains isolated context through local circular buffers (C messages) and vector-indexed long-term memory.
- Users create focused "subchats" by selecting specific text fragments from parent conversations.
- Follow-up prompts inject selected context as system messages, guiding the LLM's attention.
- Parallel reasoning threads coexist without interference, and rolling summarization preserves semantic continuity beyond the buffer horizon.

1.2 Objectives

1. To define Subchat Trees as a practical conversation architecture that combines hierarchical branching, three-tier memory management (local buffer, rolling summary, global vector store), and explicit follow-up mechanisms.
2. To construct and evaluate a 309-turn VBRI benchmark sourced from five LMSYS-Chat-1M conversations [19], covering 59 normalized topic labels and 43 topic-recall probes.
3. To analyze buffer-size behavior across $C \in \{5, 10, 20, 40\}$ and across 7B, 14B, and 32B model scales, showing that simply increasing available context does not guarantee better performance, whereas context engineering and topic isolation strongly influence accuracy, recall, and token usage.

Section 2 reviews prior work on dialogue modelling, structured reasoning, memory, retrieval, disentanglement, and multi-turn interaction. Section 3 describes the system architecture and evaluation methodology. Section 4 reports the experimental results, and Sections 5 and 6 discuss implications and conclusions.

2 Related Work

Modern work on multi-turn LLM systems builds on a longer line of sequence modeling research. Early encoder-decoder models showed that end-to-end neural systems could map one sequence to another within a unified framework [21]. Attention mechanisms then reduced the bottleneck of fixed-length sequence representations [22], and the Transformer made attention the dominant architecture for large-scale language modeling and generation [23]. With scale, autoregressive models began to exhibit broad in-context task adaptation, most visibly in GPT-3's few-shot results [24]. Instruction tuning and alignment work then pushed these systems closer to practical task-following assistants by improving zero-shot generalization and adherence to user intent [25, 26]. Against that background, current research on long-context and multi-topic dialogue focuses less on basic language generation and more on how context should be organized, retrieved, and controlled during interaction.

2.1 Background: End-to-End Dialogue Modelling

Before the current wave of instruction-following LLM systems, dialogue modelling was often approached through end-to-end generative architectures trained on conversational corpora. Hierarchical recurrent encoder-decoder models such as HRED introduced a two-level structure in which token sequences formed utterances and utterances formed conversations [10]. This work is important because it established an early argument that dialogue benefits from explicit hierarchical organization rather than flat sequence processing alone.

The limitation of this strand, from the present perspective, is that the hierarchy remains internal to the model rather than exposed in the interaction design. Users do not explicitly branch topics, separate competing threads, or decide how context should be segmented. Subchat Trees extends the same intuition about hierarchy outward, treating conversation structure as a visible and controllable part of the interface rather than only a latent modelling choice.

2.2 Structured Reasoning and Branching Search

Chain-of-Thought prompting shows that exposing intermediate reasoning steps can improve performance on tasks that require multi-step inference [3]. Self-Consistency extends this idea by sampling multiple reasoning paths and aggregating them rather than trusting a single chain [4]. Tree-of-Thought and Graph-of-Thought push the same principle further by representing reasoning as a branching or graph-structured search process rather than a single linear derivation [5, 6].

Taken together, these methods support a broader conclusion: structured representations can improve robustness, interpretability, and search efficiency. Their scope, however, is usually limited to single tasks or single prompts. They do not specify how multi-topic conversational history should be segmented over time. Subchat Trees adopts the same high-level intuition about branching and decomposition, but applies it to persistent interaction structure rather than only to reasoning traces.

2.3 Context, Memory, and Retrieval

The limitations of long conversational history are now well documented. "Lost in the middle" shows that language models exhibit strong position bias, often underusing evidence placed away from the beginning or end of the prompt [1]. LiMuT extends this concern from static prompts to multi-turn dialogue, showing that factual consistency degrades as conversations grow and the model must preserve earlier details across many exchanges [2]. These findings help explain why simply increasing context length does not reliably solve long-running dialogue problems.

Long-term dialogue benchmarks reach similar conclusions from the generation side. Beyond Goldfish Memory studies multi-session open-domain conversation and finds that models trained on short exchanges struggle when dialogue depends on information learned across sessions [16]. Long Time No See! shows that long-term persona memory remains difficult to maintain without explicit memory management [17]. LoCoMo provides a large benchmark for very long-term conversational memory and reports that even long-context and RAG-enabled systems remain far from human performance on dialogue-grounded recall and event tracking [15]. Stateful memory-augmented transformers likewise target efficient retention of dialogue history without replaying the entire conversation at each turn [12].

Memory and retrieval methods provide a complementary line of work. Retrieval-Augmented Generation (RAG) supplements parametric memory with document retrieval [13]. MemGPT frames long-context management as a hierarchical memory problem with movement between short-term context and archival storage [11]. LongLLMLingua targets efficiency by compressing prompts while preserving important information [18]. These methods improve memory use, retrieval, or prompt efficiency, but they generally still operate over a single interaction thread. Subchat Trees is complementary to them because it addresses not only what information should be stored or retrieved, but also how the conversation itself should be partitioned.

2.4 Conversation Structure and Disentanglement

Conversation disentanglement research studies how multiple threads become interleaved inside a shared message stream. Elsner and Charniak introduced an early corpus and algorithmic formulation for recovering detached conversations from mixed chat logs [7]. Jiang et al. [8] later showed that representation-learning approaches improve thread recovery by modeling message similarity within local windows. Kummerfeld et al. [9] substantially expanded the scale and quality of annotated data, helping establish disentanglement as a robust research area.

These studies are closely related to the problem addressed here, but the intervention point is different. Disentanglement is reactive: structure is inferred after messages have already been mixed. Subchat Trees instead treats branching as an explicit user action during interaction, so the system does not need to recover thread boundaries only after contamination has occurred.

2.5 Multi-Turn Interaction and Role-Based Structure

Recent work increasingly treats multi-turn interaction as a problem in its own right rather than a side effect of stronger single-turn prompting. LiMuT, LoCoMo, Beyond Goldfish Memory, and Long Time No See! all point to the same pattern: coherence, recall, and state tracking degrade over extended dialogue unless the system actively manages long-term dependencies [2, 15–17]. The implication is that multi-turn reliability depends not only on raw model capability, but also on how conversational state is organized and revisited across turns.

CAMEL adds a related perspective from multi-agent systems [14]. By assigning roles and communication protocols, it shows that explicit interaction structure can improve coordination and task performance. Subchat Trees applies a similar principle to a single-user setting: instead of multiple agents, separate branches provide explicit structure for topic-specific continuations, follow-ups, and parallel exploration.

2.6 Research Gap

The literature consistently shows that language models benefit from structure in reasoning, memory, and interaction, that long unsegmented contexts are difficult to use reliably, and that mixed conversational threads are hard to disentangle after contamination has already occurred. Yet current systems still largely rely on a flat chronological transcript as the default interface, and prior work stops short of proposing a user-facing hierarchical conversation architecture that allows sub-conversations to branch from any message, isolates context per branch, supports parallel exploration of alternatives, and scopes memory and retrieval to compact topic-specific units. To manage conversation flow more effectively and provide better context to the LLM, a user-centric interaction design is needed rather than relying only on larger context windows or post-hoc recovery. That missing interaction layer constitutes the core research gap addressed by Subchat Trees.

3 Methodology

3.1 System Architecture

Figure 1 provides an overview of the four main components: tree-based conversation structure, local buffer management, global vector memory, and the RAG retrieval pipeline.

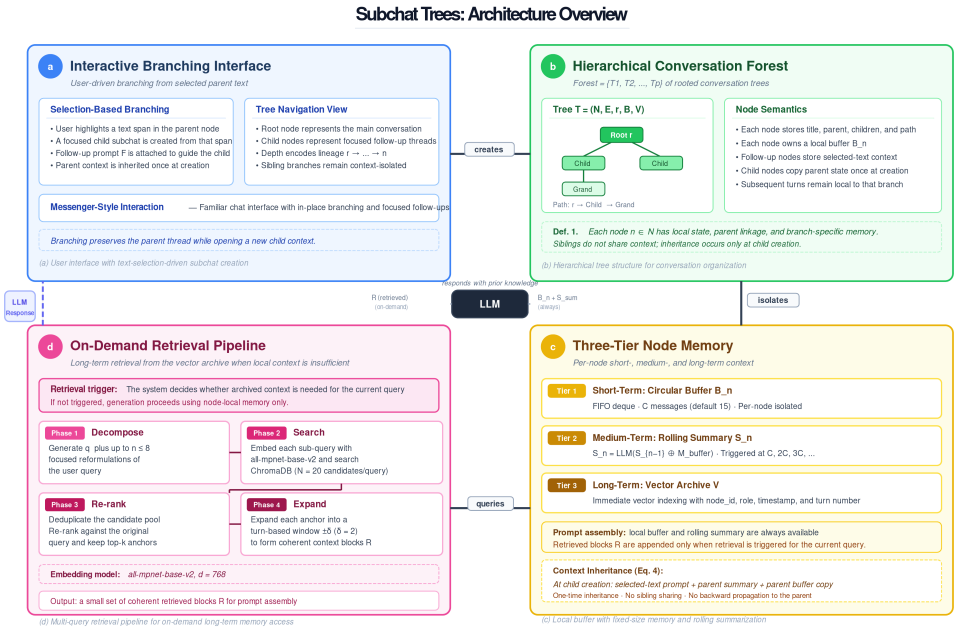


Fig. 1 Overview of the Subchat Trees methodology showing four components: (a) hierarchical tree structure for conversation organization, (b) local buffer with fixed-size memory management, (c) global vector store for long-term archival, and (d) multi-query RAG pipeline for context retrieval.

3.1.1 Hierarchical Conversation Tree Structure

Our system organizes conversations as hierarchical tree structures, where each node represents a distinct conversation thread with isolated context.

Definition 1 (Conversation Tree) A conversation tree T is defined as a tuple $(N, E, r, \mathcal{B}, \mathcal{V})$ where:

- $N = \{n_1, n_2, \ldots, n_k\}$ is the set of conversation nodes,
- $E \subseteq N \times N$ is the set of directed parent→child edges,
- $r \in N$ is the root node (main conversation),
- $\mathcal{B} : N \to B$ maps each node to its local buffer B_i,
- $\mathcal{V} : N \to \mathbb{R}^d$ maps each node's messages to d-dimensional embeddings in a shared vector store.

A `Forest` is a collection of independent trees $\mathcal{F} = \{T_1, T_2, \ldots, T_p\}$, allowing users to maintain entirely separate conversation threads (e.g., work projects, personal queries, research topics) without cross-contamination. Figure 2 illustrates this hierarchical organization as rendered in our user interface.

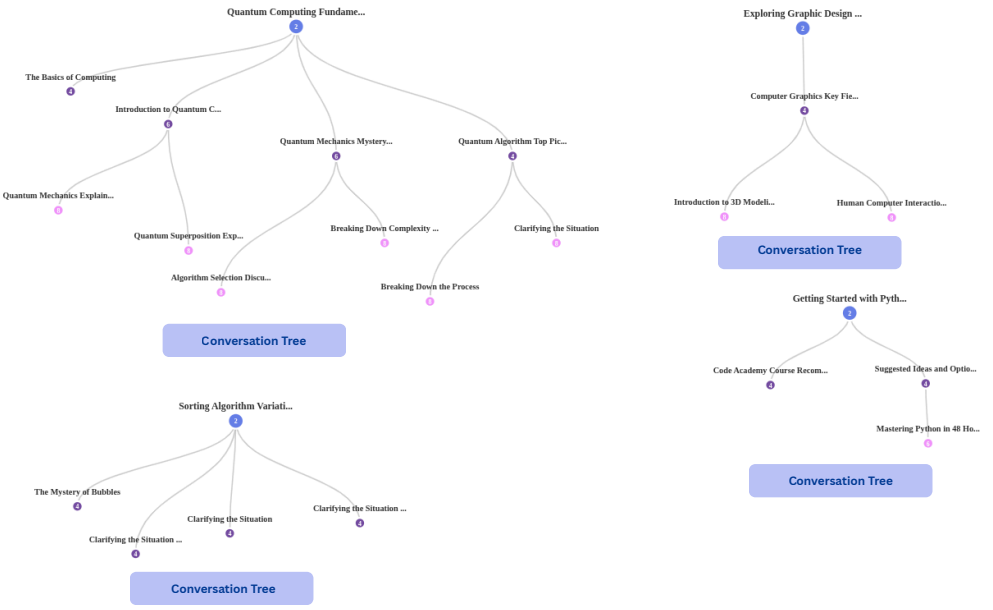


Fig. 2 Tree view of a Subchat Forest showing multiple conversation branches. Each node represents a subchat with isolated context; child nodes inherit parent context at creation time.

Each node n_i maintains:

1. **Local Buffer** B_i: a circular deque with configurable capacity C (default C=15 messages).
2. **Node Metadata:** unique identifier (`node_id`), title, parent reference, children list.
3. **Follow-up Context:** optional selected text, intent, and context type (`follow_up`, `new_topic`, or `general`).

3.1.2 Memory Management Strategy

Our memory architecture consists of three complementary tiers, illustrated in Figure 3.

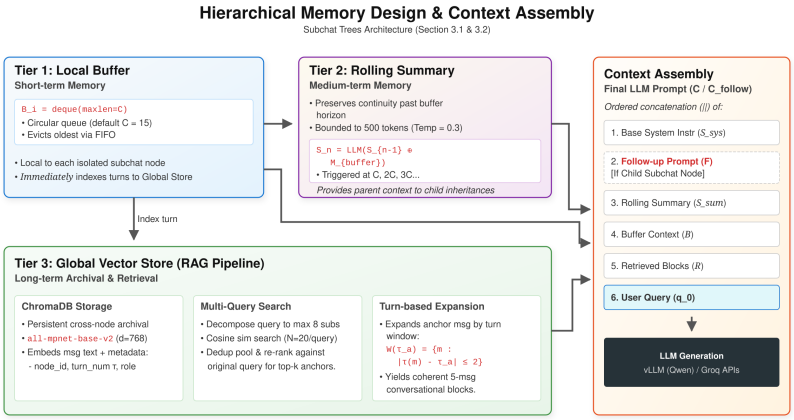


Fig. 3 Three-tier memory design showing the relationship between the LocalBuffer (circular deque), rolling summarization, and vector store archival. Messages are indexed immediately to the vector store and summarized periodically to preserve semantic continuity.

Tier 1: Local Buffer (Short-term Memory). Each node maintains a fixed-size circular buffer implemented as a Python `deque(maxlen=C)`:

$$B_i = \{(m_1, t_1), (m_2, t_2), \ldots, (m_C, t_C)\} \tag{1}$$

where m_j is the j-th message, t_j its timestamp, and C the capacity. When $|B_i| > C$, the oldest message is automatically evicted via FIFO policy. Crucially, messages are indexed to the global vector store *immediately* upon addition (not upon eviction), ensuring no information is lost during buffer turnover.

Tier 2: Rolling Summarization (Medium-term Memory). To preserve semantic continuity beyond the buffer horizon, rolling summarization is triggered whenever the total message count reaches a multiple of the buffer size:

$$S_n = \text{LLM}_{\text{sum}}(S_{n-1} \oplus M_{\text{buffer}}), \quad \text{triggered at counts } C, 2C, 3C, \ldots \tag{2}$$

where S_n is the current summary, S_{n-1} the previous summary, M_{buffer} the complete set of messages currently in the buffer, and $\oplus$ denotes concatenation. This dynamic triggering adapts to any buffer size without hard-coded thresholds. The summary is bounded at 500 tokens and generated at temperature T=0.3.

Tier 3: Global Vector Store (Long-term Memory). All messages are embedded using `all-mpnet-base-v2` (SentenceTransformers, d=768) and stored in ChromaDB with metadata:

$$\mathcal{V} = \{(\mathbf{e}(m),\ \text{meta}(m)) \mid m \in \text{all messages}\} \tag{3}$$

where $\mathbf{e}(m) \in \mathbb{R}^{768}$ is the embedding and meta(m) includes `node_id`, `timestamp`, `role`, and `conversation_title`. Retrieval uses cosine similarity with optional node-scoped filtering.

3.1.3 Context Inheritance

When creating child node n_c from parent n_p:

$$\text{Context}(n_c) = \text{FollowUpPrompt}(n_c) \cup \text{BufferCopy}(n_p) \cup \text{SummaryInherit}(n_p) \quad (4)$$

Specifically:

1. **Buffer Copy:** All messages from the parent's buffer are copied into the child's buffer, providing immediate conversational context.
2. **Summary Inheritance:** The parent's rolling summary is inherited via `inherit_summary()`, preserving long-term context that may have been evicted from the buffer.
3. **Follow-up Prompt:** The selected text and intent are injected as a system message guiding the LLM's attention.

The follow-up prompt is constructed as:

$$\text{FollowUpPrompt} = \texttt{``User selected: ''} \,\|\, s_{\text{sel}} \,\|\, \texttt{`` Focus on this.''} \quad (5)$$

Critically, subsequent messages in the child subchat do *not* propagate back to the parent, achieving bidirectional context isolation.

3.1.4 Follow-up Mechanism

Algorithm 1 formalizes the subchat creation process.

Algorithm 1 Follow-up Subchat Creation

Require: Parent node n_p, selected text s_{sel}, user query q, buffer size C
Ensure: Child node n_c with isolated context

$n_c \leftarrow \textsc{NewNode}(\text{parent}{=}n_p,\ C)$
$n_c.\text{context} \leftarrow (s_{\text{sel}},\ \texttt{``follow\_up''})$
$\text{sys\_msg} \leftarrow \textsc{BuildFollowUpPrompt}(s_{\text{sel}})$
for each message $m \in n_p.\text{buffer}$ **do**
    $n_c.\text{buffer}.\textsc{Add}(m.\text{role},\ m.\text{text})$ ▷ Inherit parent context
end for
$n_c.\text{summary} \leftarrow n_p.\text{summary}$ ▷ Inherit rolling summary
$r \leftarrow \text{LLM}(\text{sys\_msg} \,\|\, n_c.\text{buffer} \,\|\, q)$
$n_c.\text{buffer}.\textsc{Add}(\texttt{``user''},\ q)$
$n_c.\text{buffer}.\textsc{Add}(\texttt{``assistant''},\ r)$
return n_c

3.2 Vector Memory and RAG Pipeline

The global vector store serves as long-term memory, enabling retrieval of relevant context from any point in any conversation.

3.2.1 Multi-Query Decomposition and Retrieval

A single user query rarely captures all retrievable facets of a topic. Our retrieval pipeline therefore applies a four-phase process—sub-query generation, embedding-based vector search, deduplication, and re-ranking—before handing candidates to the downstream turn-based expansion stage. Figure 4 illustrates the full flow.

Sub-Query Generation.

We prompt the LLM (Groq API, temperature T=0.3) with intent-specific templates to decompose q into a set of sub-queries $Q = \{q_0, q_1, \ldots, q_{n-1}\}$, where $q_0 = q$ is the original query and $q_1 \ldots q_{n-1}$ are diverse reformulations targeting related concepts, specific entities, or broader context ($n \leq 8$).

Embedding and Vector Search.

Each sub-query q_i is independently embedded using the `all-mpnet-base-v2` sentence-transformer model [27], producing a dense vector $e(q_i) \in \mathbb{R}^{768}$. Every indexed message in the system has been embedded with the same model and stored in a persistent ChromaDB [28] collection alongside metadata fields: `node_id`, `role`, `timestamp`, `turn_number` τ, `title`, and an `archived` flag. A cosine-similarity search retrieves N=20 candidate messages per sub-query from the collection.

Deduplication.

Because overlapping sub-queries may return the same message, we deduplicate per sub-query (by content and message identifier), retaining at most $u = 5$ unique results each. A shared *seen* set further removes cross-query duplicates. The candidate pool is then:

$$\mathcal{P} = \bigcup_{i=0}^{n-1} R_i, \quad |R_i| \leq u, \quad |\mathcal{P}| \leq n \cdot u \tag{6}$$

where R_i denotes the unique results for sub-query q_i. A cosine-similarity re-ranking scores every candidate in $\mathcal{P}$ against the original query q_0 and returns the top-k anchor messages ($k \in [3, 10]$, default 5) for downstream turn-based expansion (Section 3.2.2).

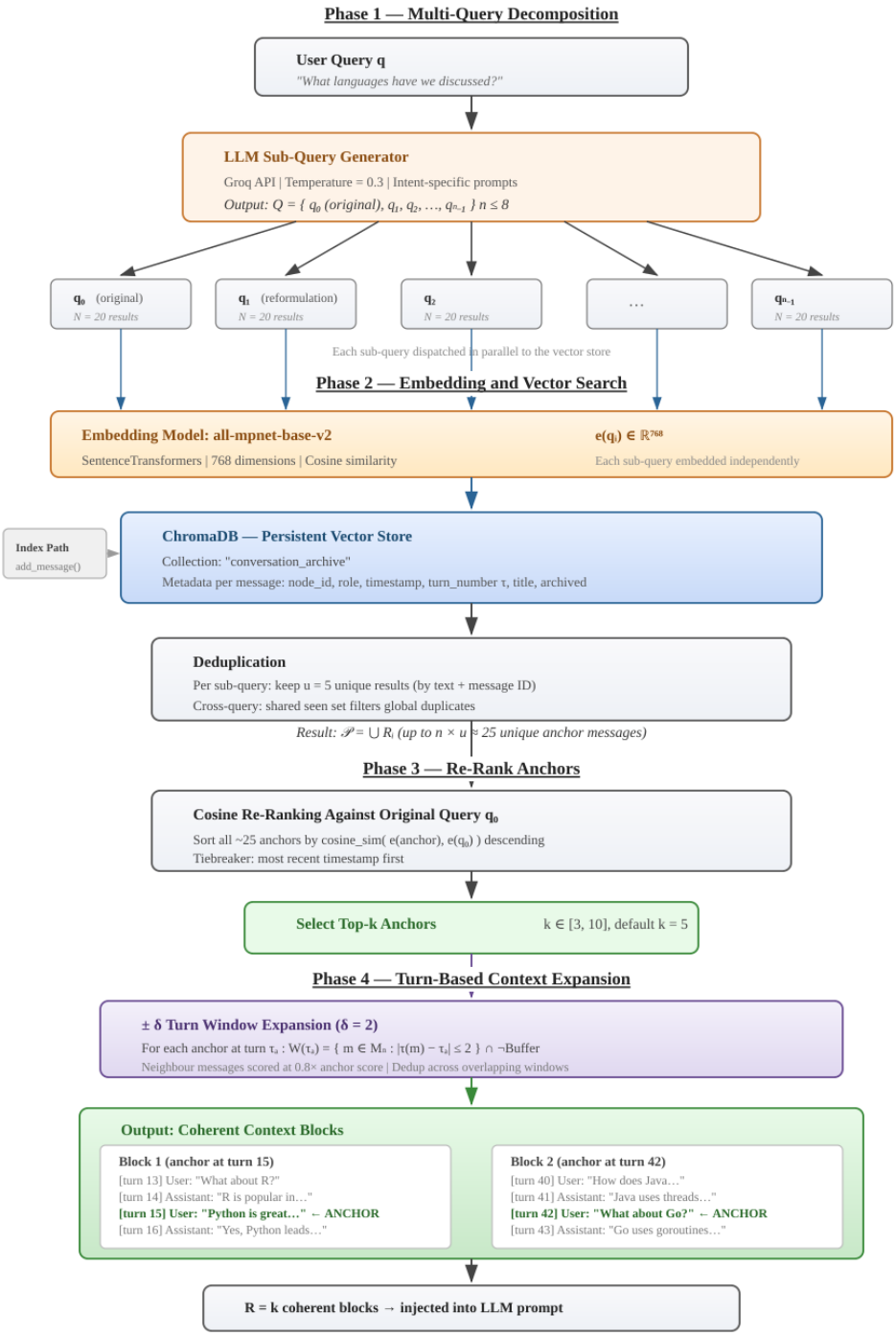


Fig. 4 End-to-end retrieval pipeline. **Phase 1**: LLM-based sub-query generation decomposes the user query into up to $n \leq 8$ sub-queries. **Phase 2**: each sub-query is embedded via `all-mpnet-base-v2` (d=768) and dispatched to ChromaDB for cosine search (N=20 candidates each), then deduplicated to u=5 unique results per sub-query. **Phase 3**: all anchor candidates are re-ranked by cosine similarity to the original query q_0; top-k are selected. **Phase 4**: each winning anchor expands $\pm\delta$ turns (δ=2) into a coherent conversational block (Eq. 7). The resulting k blocks are injected into the LLM prompt.

3.2.2 Context Window Retrieval

A retrieved message in isolation often lacks the conversational context necessary for coherent understanding. A naïve approach would define a temporal window (e.g., $\pm 60\,\mathrm{s}$ around the anchor), but user response latencies are inherently unpredictable—a reader may pause for seconds or minutes before replying—making time-based windows unreliable. We therefore define a *turn-based* context window anchored on conversational sequence rather than wall-clock time.

Each message m is indexed with a monotonically increasing turn number $\tau(m)$ within its node. For an anchor message with turn number τ_a, the context window is:

$$W(\tau_a) = \{m \in M_n : |\tau(m) - \tau_a| \leq \delta\} \tag{7}$$

where M_n is the set of all indexed messages in node n and $\delta = 2$ (i.e., two turns before and two turns after the anchor). This yields at most $2\delta + 1 = 5$ messages per anchor regardless of the time elapsed between turns. As with temporal windows, we exclude any message still held in the active buffer:

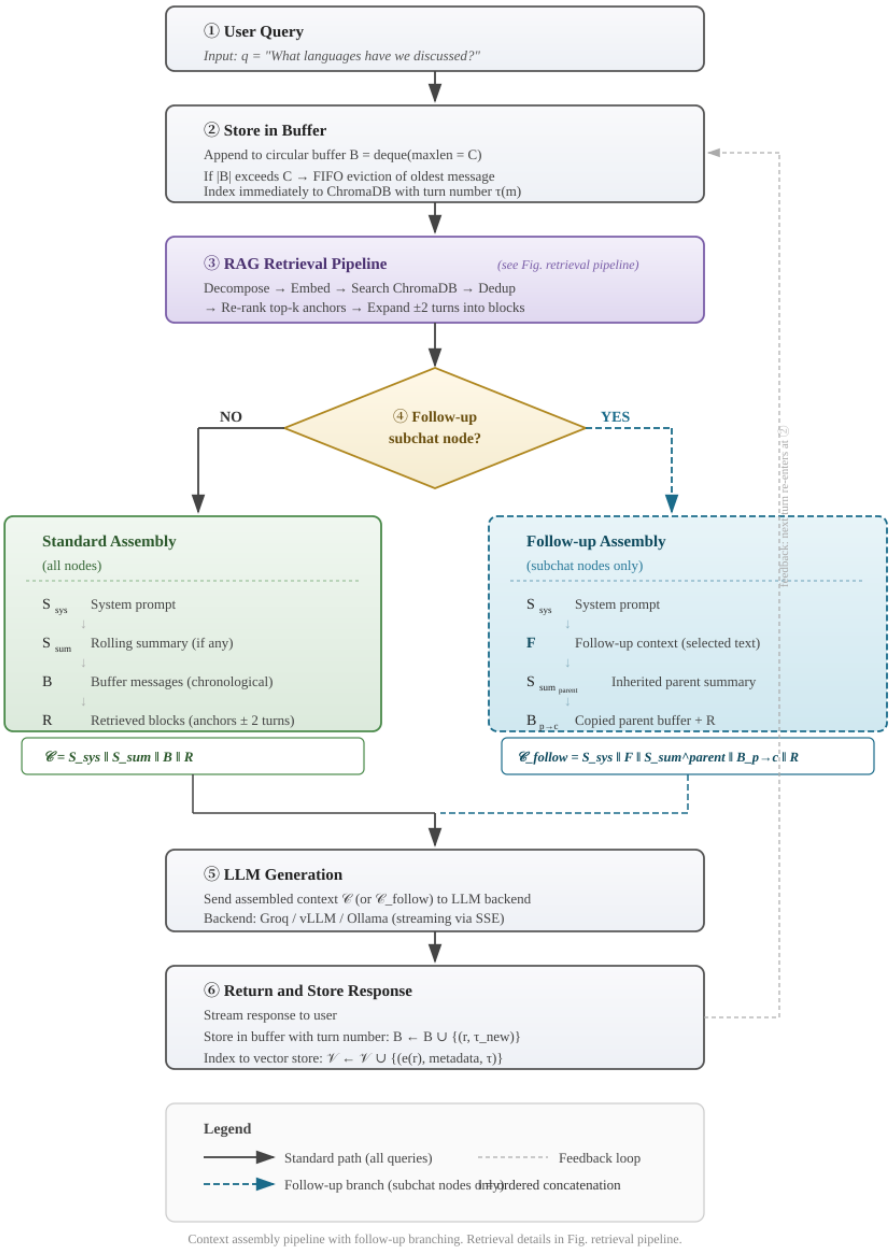
$$W_{\text{filtered}}(\tau_a) = W(\tau_a) \setminus B \tag{8}$$

where B denotes the current buffer contents.

Critically, context expansion is applied *after* re-ranking rather than before. The retrieval pipeline first re-ranks all deduplicated anchor candidates by cosine similarity to the original query q_0 and selects the top-k anchors ($k \in [3, 10]$, default 5). Only then does each winning anchor expand into its $\pm\delta$ turn window. This *rank-then-expand* ordering ensures that context neighbours never compete with anchors during ranking, and each anchor is delivered to the LLM as a coherent conversational block rather than as scattered fragments interleaved with unrelated messages. Neighbour messages receive a discounted relevance score of $0.8\times$ the anchor score, preserving the distinction between directly matched content and its surrounding narrative.

3.2.3 Context Assembly

The final prompt sent to the LLM is an ordered sequence of messages. Its composition differs depending on whether the active node is a *root-level* conversation or a *follow-up subchat*; Figure 5 illustrates both paths.



Context assembly pipeline with follow-up branching. Retrieval details in Fig. retrieval pipeline.

Fig. 5 Context assembly pipeline. The user query is buffered (①–②) and passed through the RAG retrieval pipeline (③; detailed in Figure 4), which returns coherent context blocks R. At step ④ the pipeline branches: root-level nodes follow the *standard* path (solid arrows, Eq. 9), while follow-up subchat nodes additionally inject the selected-text prompt F and the inherited parent summary $S_{\text{sum}}^{\text{parent}}$ (dashed arrows, Eq. 10). Both paths converge at step ⑤ for LLM generation, after which the response is stored back in the buffer and vector store (⑥).

Standard Assembly (all nodes).

Every query passes through the seven-step RAG pipeline described above. The assembled prompt takes the form:

$$\mathcal{C} = S_{\text{sys}} \parallel S_{\text{sum}} \parallel B \parallel R \tag{9}$$

where S_{sys} is the base system instruction, S_{sum} is the node's rolling summary (omitted if none exists), B is the circular buffer (recent turns in chronological order), R is the set of retrieved messages returned by the RAG pipeline (empty when retrieval is not triggered), and $\parallel$ denotes ordered concatenation.

Follow-up Assembly (subchat nodes).

When a user selects text in a parent conversation and creates a subchat, the child node inherits three artefacts from its parent at creation time:

1. **Selected-text prompt.** The text the user highlighted is wrapped in a system message directing the model to focus on that specific topic.
2. **Inherited summary.** The parent's rolling summary is copied to the child, giving it accumulated knowledge of the parent conversation.
3. **Parent buffer messages.** All messages currently in the parent's circular buffer are copied into the child's buffer. This ensures the child begins with the full recent context of the parent conversation.

The resulting prompt becomes:

$$\mathcal{C}_{\text{follow}} = S_{\text{sys}} \parallel F \parallel S_{\text{sum}}^{\text{parent}} \parallel B_{\text{parent}\to\text{child}} \parallel R \tag{10}$$

where F is the follow-up context message, $S_{\text{sum}}^{\text{parent}}$ is the inherited parent summary, and $B_{\text{parent}\to\text{child}}$ denotes the child's buffer initialised with the parent's messages. As the child conversation progresses, new turns are appended to this buffer (evicting older ones via FIFO), and its own rolling summary gradually replaces the inherited one, ensuring continuity while allowing topic divergence.

3.3 User Interface

Design Rationale.

The interface deliberately adopts a messenger-style layout—a familiar paradigm shared by WhatsApp, iMessage, and Slack—so that users can transfer existing mental models to the new system without additional learning overhead [29]. By anchoring interaction in a well-understood pattern (send a message, receive a reply), the cognitive cost of the novel *subchat* mechanism is kept low.

The Problem with Linear Chat.

Conventional LLM chat interfaces present all exchanges in a single, chronologically ordered thread. As conversations grow in length or branch across topics, this layout introduces several usability failures: (i) users must scroll extensively to relocate earlier

context, (ii) there is no affordance for branching—returning to a prior topic forces the user to re-state context or risk contaminating the current thread, and (iii) the model's fixed context window silently evicts earlier turns, creating an invisible boundary between what the user remembers and what the model can still "see." These issues compound in exploratory or multi-topic sessions.

Subchat Trees as a Solution.

Our interface exposes the underlying tree structure through two complementary views (Figure 6): a collapsible *tree navigator* on the left and a focused *chat panel* on the right. Users create a subchat by selecting any text in the chat panel and clicking a single button; the new subchat inherits relevant context (Section 3.2.3) without disrupting the parent conversation. This design follows the principle of *progressive disclosure* [30]: the default view is a simple chat, but richer structure (branching, parallel follow-ups, hierarchical navigation) is available on demand.

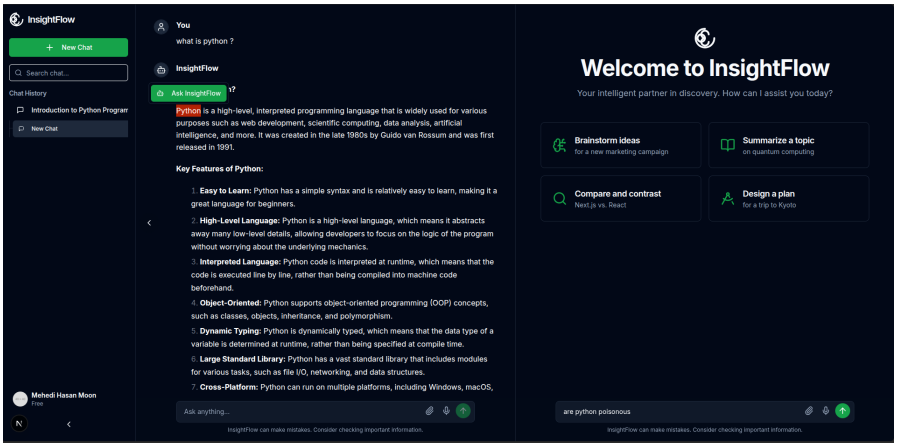


Fig. 6 User interface: (left) the conversation forest tree view with expandable nodes, and (right) the chat panel with text selection for one-click subchat creation.

Key affordances include: (i) breadcrumb navigation showing the path from root to the active subchat, (ii) visual indicators for inherited context and active retrieval status, (iii) parallel follow-ups that let users explore multiple branches of a topic simultaneously, and (iv) real-time token streaming via Server-Sent Events across multiple LLM backends (Groq, vLLM, Ollama).

3.4 Baseline System

For comparison, we implement a linear chat baseline:

- Single conversation thread with a shared circular buffer (C messages, FIFO eviction).
- All topics share the same context window—no isolation mechanism.
- Identical LLM backend, embedding model, and temperature settings.

Both systems use the same buffer capacity C in each experiment, isolating the effect of conversation structure.

3.5 Evaluation Framework

3.5.1 Dataset Design

The final evaluation dataset is `merged_realistic_interleaved.json`, a 309-turn stress-test conversation derived from five real LMSYS-Chat-1M conversations [19] produced by vicuna-13b, alpaca-13b, and koala-13b. Each source conversation was first transformed into a replayable hierarchical script with explicit topic labels, subchat actions, selected-text spans, and required response-prefix labels. The final benchmark then merges these five scripts into a single interleaved conversation.

Variable-Block Randomized Interleaving (VBRI). To stress-test context isolation and recall under realistic multi-topic switching patterns, we developed a merged evaluation scenario using Variable-Block Randomized Interleaving (VBRI). VBRI simulates sessions where users return to earlier topics after short bursts of unrelated discussion while preserving the temporal logic of each original conversation.

The VBRI algorithm operates as follows:

1. **Source-Level FIFO:** Each of the five source scenarios maintains a local queue of ordered blocks. On each iteration, the algorithm selects one source scenario and removes the next block from that source's queue. This guarantees that the original temporal order within every source conversation is preserved.
2. **Variable Blocks:** Consecutive same-topic turns are grouped into blocks. Blocks longer than $b_{\max}$=4 are split into smaller ordered sub-blocks, preserving short local coherence while preventing one topic from dominating the merged stream.
3. **Temporal Preservation:** Within each source scenario, turns maintain their original sequential order. This preserves logical dependencies (e.g., follow-up questions must appear after their referents) while allowing cross-scenario interleaving.
4. **Topic Return Dynamics:** With probability p_{ret}=0.3, the next block is drawn from the same source as the previous block; otherwise, another active source is selected uniformly. This creates bursty topic revisitation rather than complete random shuffling.
5. **Deterministic Reproducibility:** A fixed random seed (42) ensures identical interleaving across experimental runs.

The resulting merged dataset contains 309 original conversation turns spanning 59 distinct LMSYS-derived topics. It contains 91 topic switches, 64 source-conversation switches, 25 topic returns, and no temporal-order violations within any source conversation.

Summarization Probes. After the 309 ordinary turns, the dataset appends 43 summarization-probe steps, producing 352 total steps. These probes are added only for topics with at least three user turns. Each probe asks the model to summarize what was discussed about one topic, for example: `Summarize everything we have discussed about ai chat tools. Include all key points, details, and questions that were raised.` The purpose is to

test topic-level context retention after a long interleaved conversation, not only immediate topic-prefix selection. In the linear baseline, every probe is sent to the same main conversation node, where all topics have been mixed into one shared history. In Subchat Trees, probes are routed to the relevant topic subchat when available, so the model retrieves from a branch whose local buffer, summary, and vector memory are focused on one topic. This design directly tests whether dividing roles across topic-specific nodes improves the model's ability to recall earlier information.

Dataset Structure. The dataset is encoded as a JSON file with the following schema:

- `scenario_name`: Unique identifier for the scenario.
- `conversations`: Ordered list of conversation steps.
- Each step contains:
  - `step`: Integer turn number (1-indexed).
  - `context`: Ground-truth topic label (e.g., ``indian_history'', ``astrology'').
  - `message`: Raw user query.
  - `node_type`: Target subchat identifier (e.g., ``main'', ``subchat_indian_history'').
  - `action`: Operation type (`create_subchat`, `switch_node`, or implicit continuation).
  - `selected_text`: (Optional) Text fragment selected for follow-up subchat creation.
  - `subchat_title`: (Optional) Human-readable title for new subchats.
  - `parent_node_type`: (Optional) Parent node for hierarchical subchat creation.
  - `is_recall_probe`: (Optional) Boolean marker for appended summarization-probe steps.
  - `probe_topic`: (Optional) Topic being tested by a summarization probe.
  - `probe_target_node`: (Optional) Target node for a summarization probe.

For example, a step from the merged interleaved dataset:

```
{
  "step": 45,
  "context": "indian_legal_family",
  "message": "Can you explain the Hindu Succession Act?",
  "node_type": "subchat_indian_legal_family",
  "action": "create_subchat",
  "selected_text": "Indian legal framework",
  "subchat_title": "Hindu Succession Act Discussion",
  "parent_node_type": "subchat_indian_legal"
}
```

This structure enables deterministic replay: the test harness iterates through steps sequentially, performs the specified action (create subchat, switch node), sends the user message to the target node, and scores the response against the ground-truth context label.

Summarization probes use the same replay mechanism. They are encoded as `switch_node` steps so the baseline can ask the probe in its single main node, while

Subchat Trees can ask the same probe in the topic-specific node indicated by `probe_target_node`. Probe responses are excluded from topic-prefix isolation counts and are used only for recall scoring.

In the baseline condition, all steps are replayed into one linear conversation node regardless of their annotated `node_type`. In the Subchat Trees condition, `create_subchat` steps instantiate child nodes under the annotated parent, `switch_node` steps route the next message to an existing node, and ordinary continuation steps remain in the current annotated node. This means both conditions receive the same user messages in the same order; only the context organization differs.

Table 1 summarizes the dataset provenance using the original LMSYS conversation identifiers. The table reports the five source conversations that form `merged_vbri_temporal`, the final dataset used in the Kaggle run.

Table 1 Conversation-level provenance for the final VBRI dataset. Turns denote replay turns contributed to `merged_realistic_interleaved.json`, excluding appended recall probes.

Source Conversation ID	Turns	Topics	Source Stress Pattern
22807e655dd04 2348cb0ee402 3672e70	52	4	Persona roleplay, role reversal, LLM-knowledge questions, and Twenty Questions game-state confusion
6c4992f0aed04 dd3bf9a4bc225 bb4fb0	62	11	Medical discussion, AI consciousness, repetitive-loop failure, pandemic safety, and literature analysis
8d10c143f8fc4 a7599a5a1877 8fec112	77	17	DSP, C programming, medical genetics, Linux audio, physics, science fiction, geography, and games
eaf06f12a1d74 e5ca30a7ca94 a7c4128	53	8	Cookie recipes, recipe-halving math errors, argumentative correction, jokes, and emoji game turns
ec366fd3e4b54 82e8acac750f 9b3b55b	64	19	Explicit context resets across Indian history, LSAT reasoning, law, nutrition, astrology, travel, therapy, and chess
`merged_vbri_temporal`	**309**	**59**	**Final interleaved benchmark used for context-isolation scoring and recall probes**

The final replay contains 309 ordinary conversation turns and 59 normalized topic labels. Its replay annotations create 51 subchats with maximum depth 4. The probe-injected version appends 43 recall probes, producing 352 total steps; those probes are not counted as ordinary conversation turns in Table 1.

3.5.2 Evaluation Matrices

The evaluation reports four complementary matrices. Each matrix measures a different part of the same question: whether a conversation system can keep the right topic active, retain the content of that topic, do so efficiently, and avoid unnecessary retrieval from long-term memory.

Context Isolation Matrix. The context isolation matrix measures turn-level topic control. For each ordinary replay turn t, the dataset provides a ground-truth topic label τ_t. The model response is scored by extracting the leading topic prefix, denoted $\hat{\tau}_t$. A turn is correct when $\hat{\tau}_t = \tau_t$; otherwise it is counted as context pollution because the response has been assigned to the wrong conversational context. This metric therefore evaluates both memory of the active topic and the ability to match the user query to the correct topic state.

For each topic τ_i, we build a one-vs-rest confusion matrix:

- TP_i: τ_i was expected and the response was labeled as τ_i.
- FP_i: another topic was expected, but the response was labeled as τ_i.
- FN_i: τ_i was expected, but the response was labeled as another topic or as unknown.

The support n_i is the number of turns whose ground-truth topic is τ_i. Per-topic precision, recall, and F1 are:

$$P_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FP}_i}, \quad R_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FN}_i}, \quad F1_i = \frac{2P_iR_i}{P_i + R_i}. \tag{11}$$

The main table reports support-weighted averages:

$$\mathrm{Precision}_w = \sum_i \frac{n_i}{\sum_j n_j} P_i, \quad \mathrm{Recall}_w = \sum_i \frac{n_i}{\sum_j n_j} R_i, \quad \mathrm{F1}_w = \sum_i \frac{n_i}{\sum_j n_j} F1_i. \tag{12}$$

Macro averages are also computed by giving every topic equal weight. Weighted scores show performance over the full replay distribution, while macro scores check whether improvements are not limited to high-frequency topics.

Overall accuracy and pollution rate are computed over all ordinary replay turns:

$$\mathrm{Accuracy} = \frac{|\{t : \hat{\tau}_t = \tau_t\}|}{N}, \quad \mathrm{Pollution\ Rate} = 1 - \mathrm{Accuracy}, \tag{13}$$

where N is the number of ordinary replay turns. A lower pollution rate means fewer responses were contaminated by another topic.

Topic Recall Matrix. The topic recall matrix evaluates content retention rather than only topic-label selection. After the 309 ordinary replay turns, the dataset adds summarization probes for every topic with at least three user turns. Each probe asks the model to summarize what was discussed about one topic:

$$\mathrm{Probe}(\tau_i) = \textsc{SummarizeTopic}(\tau_i). \tag{14}$$

For each probed topic, the reference trace R_i is built from the user messages and assistant responses that occurred under that topic during the same evaluation run. The model's generated summary S_i is compared with R_i using ROUGE-1 F1, ROUGE-L F1, and BLEU-2:

$$\mathrm{Recall}^m = \frac{1}{|\mathcal{P}|} \sum_{\tau_i \in \mathcal{P}} m(S_i, R_i), \tag{15}$$

where $\mathcal{P}$ is the set of 43 probed topics and m is one of the three overlap metrics. ROUGE-1 measures unigram content overlap, ROUGE-L measures ordered sequence overlap, and BLEU-2 measures short phrase overlap [31, 32]. These probes are relevant because a system may output the correct topic prefix while still failing to recover the earlier details of that topic.

In the baseline, all recall probes are asked in the same main node after the full interleaved conversation. The model must therefore recover one topic from a mixed history. In Subchat Trees, probes are routed to the corresponding topic branch when such a branch exists. Since each branch stores a more focused local buffer, summary, and vector memory, the recall matrix directly tests whether topic-specific structure improves long-range retention.

System Performance Matrix. The system performance matrix measures the operational cost of producing correct answers. For each ordinary replay turn, we record input tokens, output tokens, total tokens, and latency. The average total-token cost is:

$$\overline{T}_{\text{total}} = \frac{1}{N} \sum_{t=1}^{N} T_t. \tag{16}$$

Token compression compares the average total tokens used by Subchat Trees against the linear baseline:

$$\text{Token Compression} = \frac{\overline{T}_{\text{baseline}} - \overline{T}_{\text{ours}}}{\overline{T}_{\text{baseline}}} \times 100\%. \tag{17}$$

Because a cheaper answer is not useful if it is wrong, we also report tokens per correct answer:

$$\text{Tokens Per Correct Answer} = \frac{\sum_{t=1}^{N} T_t}{|\{t : \hat{\tau}_t = \tau_t\}|}. \tag{18}$$

This combines context accuracy and token use into one efficiency measure. Cost per query is estimated from average input and output tokens:

$$\text{Cost Per Query} = \frac{\overline{T}_{\text{in}} p_{\text{in}} + \overline{T}_{\text{out}} p_{\text{out}}}{10^6}, \tag{19}$$

where p_{in} and p_{out} are prices per million input and output tokens.

RAG Decision Matrix. The RAG decision matrix measures how often the system needs to leave the active buffer and retrieve archived context. For every RAG-eligible turn, the retrieval controller returns either *search triggered* or *buffer sufficient*. Let N_{rag} be the number of eligible turns, N_{search} the number of turns that triggered retrieval, and N_{buffer} the number of turns handled by the recent buffer. We report:

$$\text{Retrieval Rate} = \frac{N_{\text{search}}}{N_{\text{rag}}} \times 100\%, \quad \text{Buffer Rate} = \frac{N_{\text{buffer}}}{N_{\text{rag}}} \times 100\%. \tag{20}$$

This matrix is relevant because retrieval behavior reflects context locality. In a flat transcript, the active buffer often contains unrelated topics, so the model may need

archived retrieval to recover the relevant history. In Subchat Trees, each subchat contains information related mainly to its own topic. A lower retrieval rate together with higher isolation or recall therefore indicates that the system is relying less on archive search because the active branch already contains more relevant context.

3.5.3 Topic-Scoring Pipeline

All scripted evaluation turns require the model to start its answer with the active topic label followed by a colon. The primary isolation score therefore uses deterministic prefix extraction rather than a subjective semantic judge. This choice makes the metric reproducible and intentionally strict: a response that discusses the correct content but begins with the wrong topic label is counted as polluted because the system failed to preserve conversational state.

Algorithm 2 Deterministic Topic-Prefix Scoring

Require: Response r, expected topic τ, valid topic set $\mathcal{T}$
Ensure: Correctness flag $y \in \{0, 1\}$ and detected topic $\hat{\tau}$

$\hat{\tau} \leftarrow \textsc{RegexExtractPrefix}(r)$ ▷ First token sequence before ":"
if $\hat{\tau} \in \mathcal{T}$ **and** $\hat{\tau} = \tau$ **then**
    return $(1, \hat{\tau})$
else
    return $(0, \hat{\tau})$
end if

For non-introductory turns, the scorer extracts the response prefix using a regular expression of the form `^([a-zA-Z_][a-zA-Z0-9_]*)\s*:`. If the extracted prefix equals the normalized ground-truth topic, the turn is counted as correct; otherwise it contributes a false negative for the expected topic and, when applicable, a false positive for the detected topic. Introductory acknowledgement turns are scored only for non-empty completion and are excluded from topic-prefix matching. An LLM-based judge module is retained in the codebase for diagnostic analysis, but the reported isolation tables use the deterministic prefix scorer.

3.6 Implementation

3.6.1 Technology Stack

- **Backend:** FastAPI (Python 3.12), stateless design for horizontal scaling.
- **LLM (Cloud):** Groq API support for hosted inference and tool-calling configurations.
- **LLM (Local GPU):** vLLM with Qwen-2.5 14B Instruct AWQ [33] (4-bit quantization) on Kaggle T4 GPUs.
- **Vector DB:** ChromaDB with `all-mpnet-base-v2` embeddings (d=768).
- **Frontend:** Next.js 14, TypeScript, with Server-Sent Events for streaming.
- **Testing:** Custom Python harness with deterministic topic-prefix scoring and ChromaDB clearing between runs.

Table 2 LLM temperature settings by task.

Task	Temperature	Rationale
Main conversation	0.0	Deterministic replay
Title generation	0.3	Consistent naming
Summarization	0.3	Structured output
Topic-prefix scoring	—	Deterministic regex evaluation
Query decomposition	0.3	Structured retrieval

3.6.2 Serverless Kaggle Evaluation Harness

A key design choice is our *serverless* evaluation harness, which enables fully reproducible experiments from a single Kaggle notebook. The traditional evaluation path routes requests through a FastAPI HTTP server; however, on GPU-constrained cloud notebooks, maintaining a separate server process alongside the vLLM model is impractical.

Our serverless runner eliminates this overhead by directly importing and instantiating the core backend classes (`SimpleChat`, `ChatGraphManager`, `Forest`, `TreeNode`, and the vector-memory components) within the same Python process as the vLLM model. The workflow proceeds as follows:

1. **Model Loading:** vLLM loads Qwen-2.5 14B Instruct AWQ with 4-bit quantization, utilizing 91% of available GPU memory (`gpu_memory_utilization=0.91`), with `max_model_len=5000` and prefix caching enabled.
2. **Registration:** The loaded model is registered globally via `VLLMClient.set_model(llm)`, making it available to all backend components for response generation and rolling summarization without network hops.
3. **Inference:** Prompts are formatted using the Hugging Face tokenizer's native `apply_chat_template()`, ensuring correct special tokens for the Qwen chat format. Single-batch offline inference via `llm.generate()` avoids HTTP serialization overhead.
4. **State Management:** Between baseline and system tests, all node maps and forest state are cleared, and the vector index collection is reset to prevent cross-condition leakage.
5. **Logging:** After each buffer size completes, the runner writes Markdown tables, raw JSON metrics, recall-probe outputs, and component logs to the buffer-specific results directory.

This architecture ensures that response generation and rolling summarization use the same in-process model, while topic-prefix scoring remains deterministic and independent of any judge model.

3.7 Experimental Setup

For each scenario, model scale, and buffer size $C \in \{5, 10, 20, 40\}$:

1. Run the linear baseline from a clean server state (fresh ChromaDB).
2. Run the Subchat Trees system from a clean server state.
3. Score all responses using deterministic topic-prefix extraction and compute per-topic metrics.

Reproducibility. Deterministic decoding for evaluation runs, deterministic topic-prefix scoring, vector-store clearing between tests, version-controlled JSON scenario files, and the single-notebook Kaggle harness ensure full reproducibility. Output length is capped during inference to standardize response length across conditions.

4 Experimental Results

The final evaluation compares a linear-chat baseline against Subchat Trees on the same 309-turn VBRI replay. We evaluate Qwen2.5 7B, 14B, and 32B AWQ models at buffer sizes $C \in \{5, 10, 20, 40\}$. The ordinary replay turns measure topic isolation through deterministic prefix scoring. After those turns, 43 recall probes ask the model to summarize prior discussion for each sufficiently frequent topic. This section is organized model by model because the most important result is not a single global score, but how model capacity changes the useful buffer range, retrieval behavior, and failure mode.

4.1 How to Read the Result Tables

The topic-isolation task uses a deliberately strict instruction. The model is told that each new topic is introduced with a pattern such as `topic_name : user query`, and that every answer must start with the active topic name followed by a colon. The key instruction is:

```
REQUIRED FORMAT for EVERY response:
Start your answer with the active topic or sub-topic name,
followed by a colon, then give your actual answer.
```

This matters because Table 1-style isolation metrics are not only semantic relevance scores. A response can discuss the right content and still be counted as polluted if it starts with the wrong topic prefix or no prefix at all. This strict scoring is useful for testing conversational state control, but it also exposes instruction-following limits in smaller models.

The recall probes use a different measurement. For each eligible topic, the probe asks:

```
Summarize everything we have discussed about {topic}. Include all
key points, details, and questions that were raised.
```

These probes are excluded from topic-prefix F1 and are reported through ROUGE-1, ROUGE-L, BLEU-2, probe token cost, and probe RAG decisions. In the baseline, every probe is asked in the single linear node. In Subchat Trees, probes are routed to the relevant topic branch when available. Therefore, recall probes test whether

the system can recover topic content after long interleaving, while isolation F1 tests whether it can keep the active topic label correct during ordinary turns.

4.2 7B Model: Context Structure Helps, but Instruction Following Remains Fragile

The 7B model exposes the hardest operating regime. Subchat Trees improves the linear baseline at every buffer size, but the model remains fragile as the prompt grows. Table 3 shows that system F1 is highest at $C = 5$ (62.9%) and then falls to 55.2%, 45.6%, and 39.3% as the buffer grows. The baseline collapses more severely, reaching only 9.8% F1 at $C = 40$. This is a model-capacity problem as much as a memory problem: the small model has difficulty preserving a hidden formatting rule after many turns, summaries, and retrieved snippets.

Table 3 7B: merged context-isolation metrics across buffer sizes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	44.7%	84.0%	71.5%	88.7%	34.9%	80.7%	21.5%	76.2%
Recall	21.1%	58.1%	39.6%	46.1%	15.6%	38.3%	7.8%	33.4%
F1	24.3%	62.9%	44.7%	55.2%	18.7%	45.6%	9.8%	39.3%
Accuracy	21.4%	58.3%	39.8%	46.3%	15.9%	38.5%	8.1%	33.7%
Pollution Rate	78.6%	41.7%	60.2%	53.7%	84.1%	61.5%	91.9%	66.3%
Macro Precision	48.5%	74.4%	70.9%	85.6%	41.7%	76.0%	29.0%	72.8%
Macro Recall	33.7%	59.6%	49.4%	59.7%	24.7%	50.5%	17.8%	41.4%
Macro F1	35.2%	60.8%	53.6%	65.3%	27.2%	55.6%	19.0%	46.5%

The recall table changes the interpretation of the low 7B F1. Table 4 shows that the tree system retains substantially more topic content than the baseline even when prefix compliance degrades. At $C = 20$, ROUGE-1 rises from 0.1811 to 0.4056, ROUGE-L rises from 0.1016 to 0.3194, and BLEU-2 rises from 0.0205 to 0.1325. In other words, the branch structure is preserving useful semantic material, but the 7B model cannot always turn that material into a correctly prefixed answer.

Table 4 7B: aggregate recall-probe scores and probe cost. Topic-specific rows are kept in the appendix; this table reports the major aggregate recall results.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.2448	0.4240	0.2369	0.4094	0.1811	0.4056	0.1557	0.3545
Avg ROUGE-L (F1)	0.1270	0.3004	0.1349	0.3097	0.1016	0.3194	0.0861	0.2574
Avg BLEU-2	0.0364	0.1581	0.0324	0.1259	0.0205	0.1325	0.0172	0.1142
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	158,043	121,521	252,519	188,722	336,942	284,817	472,634	403,852
Avg Tokens per Probe	3,675	2,826	5,873	4,389	7,836	6,624	10,991	9,392
Total Probe Latency	918.00s	710.28s	814.85s	776.91s	767.61s	792.74s	723.47s	703.08s
Avg Probe Latency	21.35s	16.52s	18.95s	18.07s	17.85s	18.44s	16.82s	16.35s

The performance table also shows why raw token count alone is not the right story. At $C = 5$, the system uses fewer average input tokens than baseline, but at larger buffers system input becomes slightly larger. This does not contradict the architecture. The logs show that disorganized baseline context often produces short, generic, or incorrectly prefixed answers, while branch-local context gives the model enough relevant evidence to answer more fully. The better efficiency signal is tokens per correct answer: Subchat Trees lowers this metric at every 7B buffer size, including 35,451 to 15,389 at $C = 20$ and 102,923 to 25,899 at $C = 40$.

Table 5 7B: merged system-performance metrics, including recall/summarization probe token accounting.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	2,270	2,205	3,384	3,556	5,386	5,685	8,107	8,487
Avg Output Tokens	205	164	214	235	235	241	220	230
Avg Total Tokens	2,476	2,369	3,598	3,791	5,622	5,927	8,327	8,717
Total Tokens	765,017	732,147	1,111,680	1,171,373	1,737,082	1,831,296	2,573,086	2,693,481
Tokens Per Correct Answer	11,591	4,067	9,038	8,191	35,451	15,389	102,923	25,899
Avg Latency	12.19s	10.14s	10.62s	10.55s	11.70s	11.22s	11.75s	11.71s
Cost per Query	$0.000130	$0.000123	$0.000186	$0.000197	$0.000288	$0.000304	$0.000423	$0.000443
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	158,043	121,521	252,519	188,722	336,942	284,817	472,634	403,852
Avg Tokens per Summarization Probe	3,675	2,826	5,873	4,389	7,836	6,624	10,991	9,392
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	923,060	853,668	1,364,199	1,360,095	2,074,024	2,116,113	3,045,720	3,097,333
Combined Avg Tokens per Call	2,622	2,425	3,876	3,864	5,892	6,012	8,653	8,799

The 7B RAG table should be read as a retrieval-calibration failure rather than as a simple win/loss count. During normal turns, retrieval is modest and drops as buffers grow. During recall probes, however, the system triggers RAG more than the baseline at every buffer size. At $C = 5$, system probe retrieval is 39.5% while baseline probe retrieval is only 2.3%. This is not the expected memory-sufficiency pattern; it suggests the smaller model is unsure whether branch-local context is enough and overuses retrieval despite stronger recall scores.

Table 6 7B: merged RAG decision breakdown for ordinary turns and recall/summarization probes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Total Turns	309	309	309	309	309	309	309	309
RAG-Eligible Turns	309	309	309	309	306	309	302	309
RAG Triggered	47	48	42	49	27	12	6	9
Buffer Sufficient	262	261	267	260	279	297	296	300
Retrieval Rate	15.2%	15.5%	13.6%	15.9%	8.8%	3.9%	2.0%	2.9%
Buffer Rate	84.8%	84.5%	86.4%	84.1%	91.2%	96.1%	98.0%	97.1%
Errors	0	0	0	0	3	0	7	0
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	43
Probe RAG Triggered	1	17	2	9	2	4	0	1
Probe Buffer Sufficient	42	26	41	34	41	39	43	42
Probe Retrieval Rate	2.3%	39.5%	4.7%	20.9%	4.7%	9.3%	0.0%	2.3%
Probe Buffer Rate	97.7%	60.5%	95.3%	79.1%	95.3%	90.7%	100.0%	97.7%
Probe Errors	0	0	0	0	0	0	0	0

Overall, the 7B finding is precise: the tree architecture improves memory and reduces pollution, but a small model still struggles to obey the strict prefix protocol in long, interleaved sessions. The architecture helps, but it cannot fully compensate for weak instruction following.

4.3 14B Model: The Cleanest Practical Tradeoff

The 14B model gives the clearest evidence for the architecture. Table 7 shows that system F1 improves from 73.9% at $C = 5$ to 83.4% at $C = 10$ and 83.1% at $C = 20$. The drop to 73.1% at $C = 40$ is important: it shows that increasing the buffer is not automatically beneficial. At $C = 40$, summarization happens later, leaving more raw user and assistant turns in the prompt together with summaries, system instructions, and possible retrieval context. The result is not simply more memory; it is a heavier prompt that can overwhelm the model.

Table 7 14B: merged context-isolation metrics across buffer sizes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	64.5%	77.8%	81.2%	93.2%	85.0%	89.2%	81.6%	85.3%
Recall	52.6%	74.4%	59.7%	80.2%	60.1%	81.8%	55.8%	69.5%
F1	51.2%	73.9%	64.3%	83.4%	62.5%	83.1%	59.8%	73.1%
Accuracy	52.8%	74.4%	59.9%	80.3%	60.2%	81.9%	56.0%	69.6%
Pollution Rate	47.2%	25.6%	40.1%	19.7%	39.8%	18.1%	44.0%	30.4%
Macro Precision	52.0%	67.5%	75.7%	88.1%	77.6%	85.4%	73.4%	79.5%
Macro Recall	52.7%	68.5%	67.3%	78.7%	65.4%	79.5%	61.8%	65.3%
Macro F1	48.3%	65.8%	67.1%	80.1%	65.1%	79.7%	61.4%	67.9%

The recall-probe table shows the same pattern from the memory side. System ROUGE-1 remains around 0.42 at $C = 10$ and $C = 20$, while baseline ROUGE-1 drops from 0.3378 at $C = 10$ to 0.2476 at $C = 20$ and 0.1745 at $C = 40$. This is the cleanest evidence that the baseline is not only forgetting; it is mixing unrelated context. The tree system keeps the relevant branch recoverable.

Table 8 14B: aggregate recall-probe scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.3428	0.3896	0.3378	0.4201	0.2476	0.4187	0.1745	0.3725
Avg ROUGE-L (F1)	0.2179	0.2634	0.1982	0.3211	0.1320	0.3143	0.0997	0.2648
Avg BLEU-2	0.0967	0.1099	0.0870	0.1338	0.0516	0.1237	0.0182	0.1315
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	136,045	104,023	274,209	156,136	322,902	269,171	359,355	304,196
Avg Tokens per Probe	3,164	2,419	6,377	3,631	7,509	6,260	8,357	7,074
Total Probe Latency	1155.49s	989.76s	1441.58s	997.15s	1284.58s	1196.07s	943.70s	957.29s
Avg Probe Latency	26.87s	23.02s	33.53s	23.19s	29.87s	27.82s	21.95s	22.26s

The 14B performance table gives the practical tradeoff. Raw input tokens are not always lower for the system, especially at $C = 20$ and $C = 40$, but tokens per correct answer are lower at every buffer size. At $C = 20$, the baseline spends 8,036 tokens per correct answer while the system spends 6,853. At $C = 40$, the baseline spends 10,418 while the system spends 8,826. The system is therefore more efficient when correctness is considered, even if the average prompt is not always shorter.

Table 9 14B: merged system-performance metrics, including recall/summarization probe token accounting.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	1,818	1,919	3,152	3,092	4,659	5,396	5,685	5,981
Avg Output Tokens	133	145	170	182	177	215	148	160
Avg Total Tokens	1,952	2,064	3,321	3,274	4,837	5,611	5,833	6,141
Total Tokens	603,033	637,814	1,026,292	1,011,686	1,494,607	1,733,862	1,802,333	1,897,579
Tokens Per Correct Answer	3,700	2,773	5,548	4,079	8,036	6,853	10,418	8,826
Avg Latency	16.71s	16.16s	15.00s	13.58s	16.06s	17.69s	14.58s	14.64s
Cost per Query	$0.000102	$0.000108	$0.000171	$0.000169	$0.000247	$0.000287	$0.000296	$0.000312
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	136,045	104,023	274,209	156,136	322,902	269,171	359,355	304,196
Avg Tokens per Summarization Probe	3,164	2,419	6,377	3,631	7,509	6,260	8,357	7,074
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	739,078	741,837	1,300,501	1,167,822	1,817,509	2,003,033	2,161,688	2,201,775
Combined Avg Tokens per Call	2,100	2,107	3,695	3,318	5,163	5,690	6,141	6,255

The RAG table provides the strongest retrieval story for 14B. During recall probes, baseline retrieval is high at small buffers and then decreases as the buffer grows: 58.1%, 44.2%, 37.2%, and 16.3%. The system is much lower: 23.3%, 2.3%, 4.7%, and 0.0%. This is the expected behavior. In the baseline, old topic content is either evicted or buried among unrelated turns, so the model asks for archive retrieval. In the tree system, the active branch often already contains enough local evidence.

Table 10 14B: merged RAG decision breakdown for ordinary turns and recall/summarization probes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Total Turns	309	309	309	309	309	309	309	309
RAG-Eligible Turns	309	309	309	309	306	309	284	291
RAG Triggered	36	29	26	29	27	30	22	21
Buffer Sufficient	273	280	283	280	279	279	262	270
Retrieval Rate	11.7%	9.4%	8.4%	9.4%	8.8%	9.7%	7.7%	7.2%
Buffer Rate	88.3%	90.6%	91.6%	90.6%	91.2%	90.3%	92.3%	92.8%
Errors	0	0	0	0	3	0	25	18
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	41
Probe RAG Triggered	25	10	19	1	16	2	7	0
Probe Buffer Sufficient	18	33	24	42	27	41	36	41
Probe Retrieval Rate	58.1%	23.3%	44.2%	2.3%	37.2%	4.7%	16.3%	0.0%
Probe Buffer Rate	41.9%	76.7%	55.8%	97.7%	62.8%	95.3%	83.7%	100.0%
Probe Errors	0	0	0	0	0	0	0	2

The 14B model therefore supports the main architectural claim most cleanly. Moderate buffers provide enough local context while allowing summarization to compact

older turns before the prompt becomes too raw and noisy. In this setting, Subchat Trees improves isolation, recall, retrieval sufficiency, and tokens per correct answer.

4.4 32B Model: Strongest Absolute Performance and Better Long-Context Tolerance

The 32B model reaches the strongest absolute scores. Table 11 shows that the system is above 83% F1 from $C = 5$ through $C = 20$, peaking at 85.2% at $C = 20$. Even at $C = 40$, the system remains at 78.2%, a smaller drop than the 7B and 14B models. This indicates that stronger models can tolerate longer context better, but the system still improves over the baseline at every buffer size.

Table 11 32B: merged context-isolation metrics across buffer sizes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	88.0%	88.5%	87.3%	94.4%	89.9%	93.4%	84.9%	90.0%
Recall	64.6%	83.1%	69.2%	80.2%	67.2%	83.4%	63.6%	74.4%
F1	68.6%	83.2%	71.1%	83.7%	69.6%	85.2%	65.7%	78.2%
Accuracy	64.7%	83.2%	69.3%	80.3%	67.3%	83.5%	63.8%	74.4%
Pollution Rate	35.3%	16.8%	30.7%	19.7%	32.7%	16.5%	36.2%	25.6%
Macro Precision	83.6%	85.1%	81.4%	91.8%	86.7%	93.0%	78.3%	88.2%
Macro Recall	73.9%	81.9%	75.2%	79.8%	75.8%	86.8%	71.6%	74.0%
Macro F1	74.4%	80.8%	73.9%	81.9%	74.5%	87.1%	68.4%	76.7%

The recall metrics are also strongest for the system. At $C = 20$, system ROUGE-1 is 0.4283 compared with 0.3338 for the baseline. At $C = 40$, the baseline collapses to 0.1503 while the system remains at 0.3918. This is one of the clearest signs that the linear baseline becomes overwhelmed by topic mixture, while the tree system keeps the relevant branch recoverable.

Table 12 32B: aggregate recall-probe scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.2687	0.4328	0.3674	0.4206	0.3338	0.4283	0.1503	0.3918
Avg ROUGE-L (F1)	0.1767	0.2830	0.2274	0.2982	0.2009	0.3016	0.0878	0.2624
Avg BLEU-2	0.0502	0.1452	0.1023	0.1253	0.0731	0.1247	0.0234	0.1206
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	119,832	131,487	249,908	175,875	373,285	264,906	324,171	310,617
Avg Tokens per Probe	2,787	3,058	5,812	4,090	8,681	6,161	7,539	7,224
Total Probe Latency	1959.92s	2495.09s	2965.46s	2722.82s	3002.40s	2796.51s	1888.54s	2288.25s
Avg Probe Latency	45.58s	58.03s	68.96s	63.32s	69.82s	65.04s	43.92s	53.22s

The 32B performance table shows the cost of stronger inference, but also confirms the same efficiency pattern. The 32B model is slower than the smaller models, yet Subchat Trees reduces tokens per correct answer at every buffer size. At $C = 20$, the

baseline spends 7,775 tokens per correct answer while the system spends 6,600. The system also has lower average latency than baseline at $C = 5$, $C = 10$, and $C = 40$, showing that organized context can offset some of the cost of richer answers.

Table 13 32B: merged system-performance metrics, including recall/summarization probe token accounting.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	2,038	2,119	3,165	3,156	5,043	5,292	6,252	6,405
Avg Output Tokens	139	158	173	176	191	219	162	170
Avg Total Tokens	2,177	2,277	3,338	3,331	5,234	5,511	6,413	6,575
Total Tokens	672,623	703,603	1,031,367	1,029,385	1,617,294	1,702,877	1,981,748	2,031,629
Tokens Per Correct Answer	3,363	2,738	4,819	4,151	7,775	6,600	10,060	8,833
Avg Latency	36.91s	35.86s	37.52s	33.71s	38.99s	39.29s	34.77s	33.56s
Cost per Query	$0.000113	$0.000119	$0.000172	$0.000172	$0.000267	$0.000282	$0.000326	$0.000334
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	119,832	131,487	249,908	175,875	373,285	264,906	324,171	310,617
Avg Tokens per Summarization Probe	2,787	3,058	5,812	4,090	8,681	6,161	7,539	7,224
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	792,455	835,090	1,281,275	1,205,260	1,990,579	1,967,783	2,305,919	2,342,246
Combined Avg Tokens per Call	2,251	2,372	3,640	3,424	5,655	5,590	6,551	6,654

The 32B RAG behavior is more nuanced than 14B. At $C = 5$, the system retrieves more during recall probes than the baseline, suggesting that a short branch-local buffer may not contain enough evidence for full-topic summaries. But once the buffer reaches 10, 20, and 40, system probe retrieval falls below baseline. At $C = 40$, baseline probe retrieval is 72.1% while system probe retrieval is only 2.4%. The stronger model can use the organized branch context effectively once enough local history is available.

Table 14 32B: merged RAG decision breakdown for ordinary turns and recall/summarization probes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Total Turns	309	309	309	309	309	309	309	309
RAG-Eligible Turns	309	309	309	309	309	309	300	306
RAG Triggered	61	80	84	66	71	67	64	48
Buffer Sufficient	248	229	225	243	238	242	236	258
Retrieval Rate	19.7%	25.9%	27.2%	21.4%	23.0%	21.7%	21.3%	15.7%
Buffer Rate	80.3%	74.1%	72.8%	78.6%	77.0%	78.3%	78.7%	84.3%
Errors	0	0	0	0	0	0	9	3
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	42
Probe RAG Triggered	23	38	40	23	26	8	31	1
Probe Buffer Sufficient	20	5	3	20	17	35	12	41
Probe Retrieval Rate	53.5%	88.4%	93.0%	53.5%	60.5%	18.6%	72.1%	2.4%
Probe Buffer Rate	46.5%	11.6%	7.0%	46.5%	39.5%	81.4%	27.9%	97.6%
Probe Errors	0	0	0	0	0	0	0	1

The 32B finding is that model capacity and context architecture are complementary. The stronger model tolerates larger buffers better, but the tree system still provides a consistent advantage in isolation, recall, pollution reduction, and tokens per correct answer.

4.5 Cross-Model Visual Synthesis

The model-specific tables show the detailed behavior. The following figures summarize the global pattern across models and buffers. Table 15 reports the best observed system setting for each model. The best absolute result is 32B at $C = 20$, while 14B at $C = 10/20$ provides a strong practical tradeoff. The 7B model shows the largest rescue effect but remains instruction-following limited.

Table 15 Best Subchat Trees configuration for each model. F1 gain is reported as absolute percentage-point improvement over the corresponding linear baseline at the same buffer size.

Model	Best C	Base F1	System F1	F1 Gain	Base Pollution	System Pollution
7B	5	24.3%	62.9%	+38.6 pts	78.6%	41.7%
14B	10	64.3%	83.4%	+19.1 pts	40.1%	19.7%
32B	20	69.6%	85.2%	+15.7 pts	32.7%	16.5%

Figure 7 shows context-isolation F1 across all runs. Subchat Trees improves over the baseline in every configuration. The weakest model benefits most in relative terms, while the strongest model reaches the highest absolute performance.

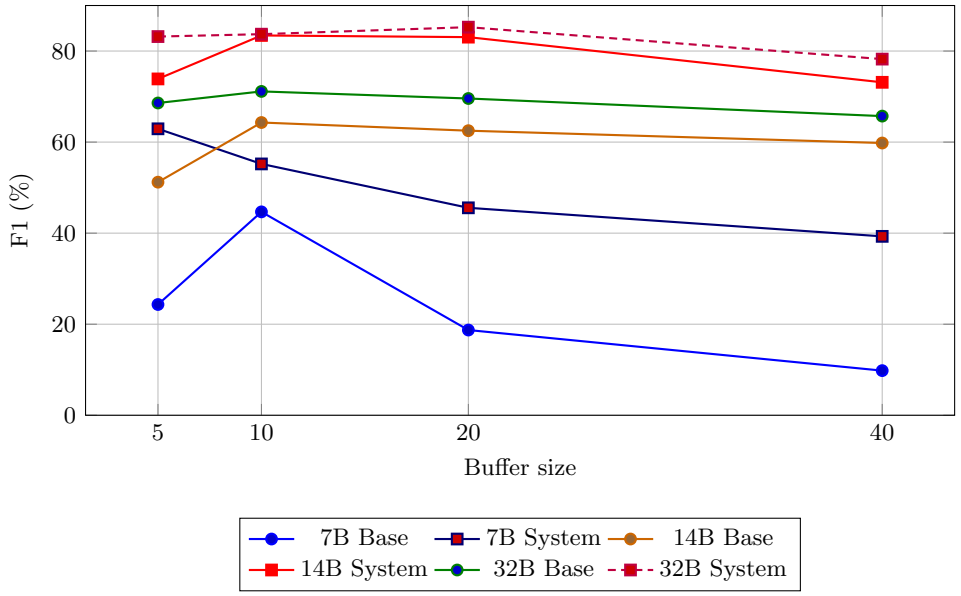


Fig. 7 Context-isolation F1 across model sizes and buffer sizes. Subchat Trees improves F1 over the linear baseline in every configuration.

Figure 8 gives the matching pollution view. At $C = 20$, pollution falls from 84.1% to 61.5% for 7B, from 39.8% to 18.1% for 14B, and from 32.7% to 16.5% for 32B. This directly supports the central claim: the failure is not only forgetting, but also contamination from unrelated conversational state.

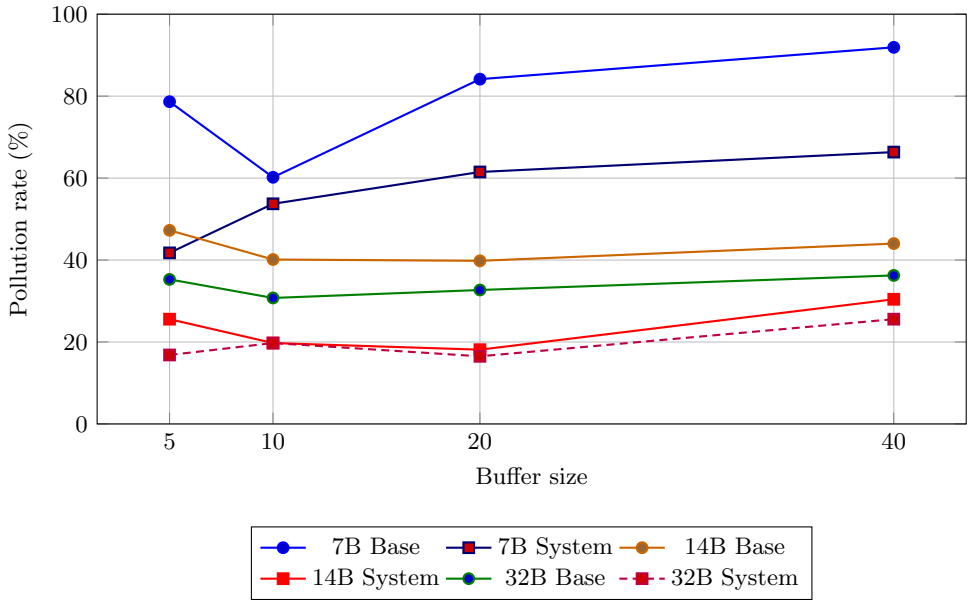


Fig. 8 Pollution rate across buffer sizes. Lower values are better. Subchat Trees consistently reduces cross-topic contamination by routing turns into branch-local buffers instead of keeping all topics in one linear transcript.

Figure 9 summarizes the recall-probe result. The system improves average ROUGE-1 in every model-buffer setting. This matters because it shows that the tree is preserving topic content, not merely making the prefix scorer happy.

Figure 10 focuses on recall-probe RAG decisions. This is the cleanest RAG visualization because each probe explicitly asks the model to recover prior topic content. The 14B model shows the expected pattern most clearly: baseline retrieval is high at small buffers, while the tree system quickly becomes buffer-sufficient. The 7B and 32B exceptions at short buffers are informative rather than contradictory: they show that retrieval decisions are also model-calibration decisions.

Finally, Figure 11 summarizes effective token efficiency. The system sometimes uses more raw input tokens, but it uses fewer tokens per correct answer in every tested setting. This is the key correction to the initial expectation: organized context is not always shorter, but it is more productive.

4.6 Result Summary

The model-specific and cross-model results support five conclusions. First, branch-local context improves topic isolation across every tested model and buffer size. Second, the useful buffer range is model-dependent: 7B is safest at short buffers, 14B works best around $C = 10$ or $C = 20$, and 32B peaks at $C = 20$. Third, recall probes show that the tree system preserves semantic topic content even when strict prefix-following fails. Fourth, recall-probe RAG is a better retrieval diagnostic than normal-turn RAG because the probe explicitly asks for old topic content. Fifth, raw input-token count

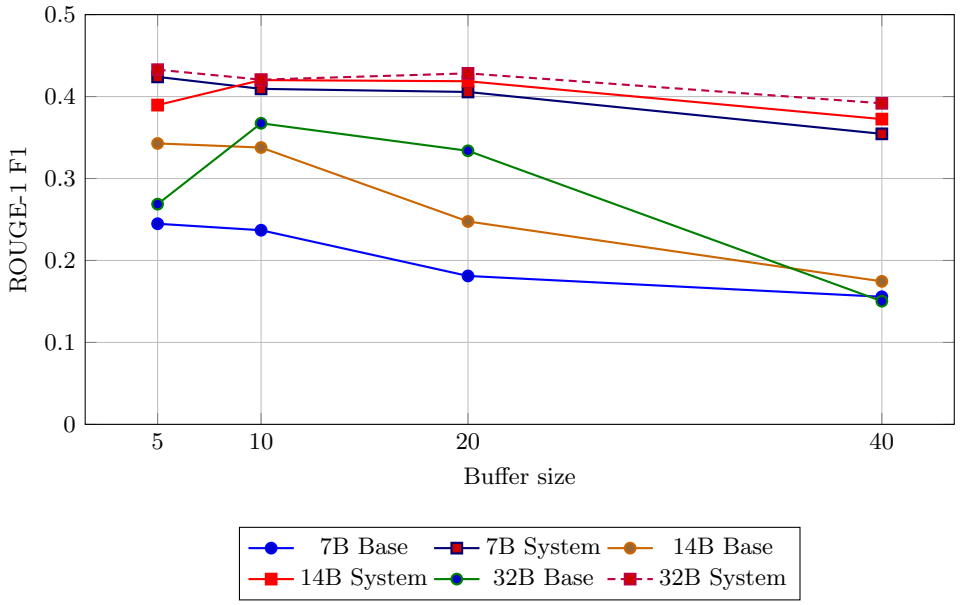


Fig. 9 Average ROUGE-1 F1 on 43 topic-recall probes. The tree system retains more semantically relevant topic content than the linear baseline across all tested model-buffer settings.

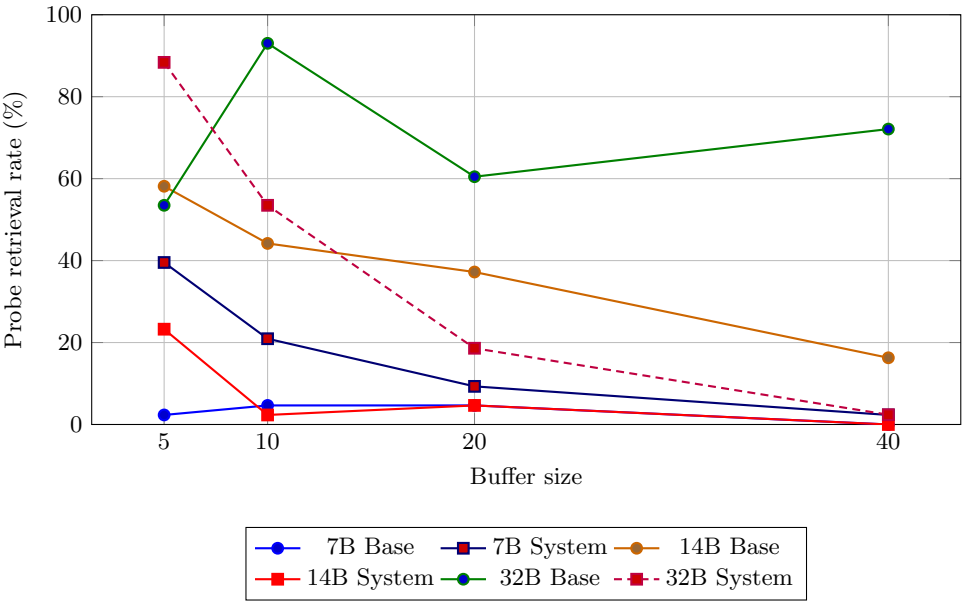


Fig. 10 RAG retrieval rate during the 43 recall probes. Probe retrieval is a cleaner memory-sufficiency signal than normal-turn retrieval because every probe asks the model to recover earlier topic content.

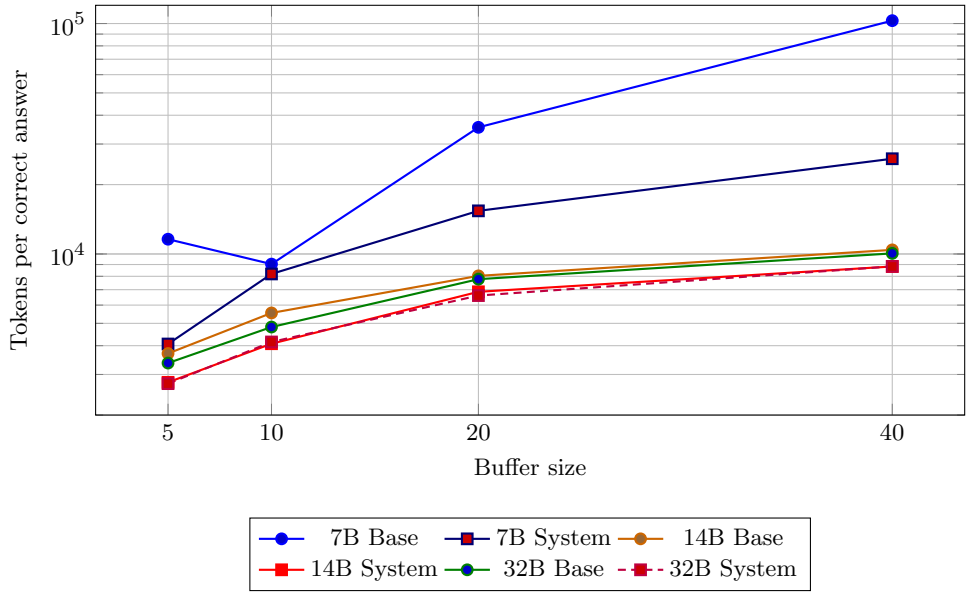


Fig. 11 Tokens per correct answer across model sizes and buffer sizes. The y-axis uses a log scale because the 7B baseline becomes extremely inefficient at larger buffers.

is not the main efficiency result; tokens per correct answer shows that Subchat Trees spends computation more productively.

These results motivate conversation trees as more than a display feature. A parent conversation can hold broad context, child nodes can isolate detailed follow-ups, and sibling nodes can explore alternatives without contaminating one another. This matters for ordinary chat, multi-agent workflows, and embodied systems where plans, observations, corrections, and subgoals naturally form nested conversational state. Long context windows increase capacity, but they do not decide which prior turns belong to the current branch. Subchat Trees makes that structure explicit.

5 Discussion

5.1 What the Results Show

The experiments show that conversation structure is not merely an interface preference; it changes model behavior. A linear transcript forces the LLM to infer which parts of history belong to the current turn. Subchat Trees externalizes this decision by routing turns into explicit branches. The result is lower topic pollution, higher F1, and stronger recall-probe similarity across every tested model-buffer configuration.

The cross-model pattern is also important. The 7B model shows that architecture cannot fully compensate for weak instruction following: even with better context retention, the model often fails the exact topic-prefix format. The 14B model shows the best practical balance in these runs, achieving high F1 with much lower latency than 32B. The 32B model shows the strongest absolute capability, especially at $C = 20$,

confirming that model scale and context architecture are complementary rather than interchangeable.

5.2 Why Larger Buffers Are Not Always Better

The buffer ablation demonstrates that more raw context can become harmful. At $C = 40$, summarization is delayed and many raw turns remain in the prompt. This increases the amount of user wording, assistant wording, old instructions, possible RAG context, and unrelated topic material that the model must reconcile. The 14B and 7B models degrade noticeably at this size, and the error counts also rise. The 32B model tolerates the longer prompt better, but even it performs best at $C = 20$ rather than $C = 40$.

This supports a core claim of the paper: context length and context organization are different problems. Long context windows increase capacity, but they do not decide which branch of a conversation should be active. Subchat Trees improves the structure of the input before the model is asked to reason over it.

5.3 Token Efficiency Revisited

The current results revise our initial token-efficiency expectation. We expected that Subchat Trees would always reduce average input tokens because the active branch should be smaller than the full linear history. In practice, the system sometimes uses slightly more raw tokens. The logs suggest a reason: a confused baseline often answers briefly, asks for clarification, or emits a generic response, while a scoped system prompt allows the model to produce longer and more complete answers.

For this reason, tokens per correct answer is the more meaningful efficiency metric. Under that metric, Subchat Trees is consistently better. It may spend more tokens on a given turn, but it spends fewer tokens for each correct topic-tracked answer. This distinction matters for real deployment because users care about useful successful answers, not only raw prompt length.

5.4 RAG as a Backstop Rather Than a Default

The recall-probe RAG analysis suggests that retrieval works best as a backstop. In the 14B model and in larger-buffer 32B settings, the baseline frequently asks for retrieval during topic summarization because older topic details have either been evicted or mixed with unrelated turns. The tree system often decides that the branch-local buffer is already sufficient. This is the desired behavior: retrieval should be used when local context is insufficient, not as a replacement for well-organized conversation state.

The 7B exception is informative. It triggers RAG more often in the system condition, especially at short buffers, indicating that the smaller model is less reliable at judging context sufficiency. This makes the retrieval controller itself model-sensitive. Stronger models not only answer better; they also make better decisions about when memory is needed.

5.5 Limitations

Strict Prefix Scoring. The deterministic scorer counts a response as incorrect if the prefix does not exactly match the expected topic. This is appropriate for testing context isolation, but it can understate semantic quality when a model answers the right content with the wrong label.

Model Family. The current cross-scale runs use Qwen2.5 AWQ models. Additional model families should be evaluated before claiming architecture-independent generality.

Buffer Search. We evaluate $C \in \{5, 10, 20, 40\}$. The best value is model-dependent, and adaptive buffer sizing may outperform any fixed value.

RAG Decision Reliability. RAG decisions depend on the model's ability to judge whether local context is sufficient. The 7B results show that this controller can be miscalibrated for weaker models.

User-Specified Branching. The benchmark uses explicit subchat actions. Real deployments should study automatic branch suggestions and user burden.

5.6 Deployment Considerations

Production deployment requires efficient vector-index sharding, branch archival policies, cache-aware context assembly, and safeguards for branch inheritance. The architecture is model-agnostic and can run over local vLLM backends, cloud APIs, or hybrid deployments. The main systems lesson is that the context manager should be treated as an active component of the LLM application, not a passive transcript buffer.

6 Conclusion and Future Work

We introduced Subchat Trees, a hierarchical conversation architecture for isolating topics in long, multi-threaded LLM dialogue. The current evaluation across Qwen2.5 7B, 14B, and 32B models shows that explicit conversation branching improves context-isolation F1 in every tested model-buffer configuration. The best absolute result is obtained by the 32B model at $C = 20$, where F1 rises from 69.6% to 85.2% and pollution falls from 32.7% to 16.5%. The 14B model reaches a similar system F1 of 83.4% at $C = 10$ and 83.1% at $C = 20$, making it the strongest practical tradeoff in these runs.

The recall probes add a second conclusion: Subchat Trees preserves semantically relevant topic content, not only topic labels. Across all models and buffers, system ROUGE-1 exceeds the baseline. Finally, the efficiency analysis shows that raw input-token count is not sufficient for evaluating context architectures. The system sometimes produces longer prompts or completions, but it consistently reduces tokens per correct answer.

Overall, the results support the paper's central claim: long conversations need structure, not only longer windows. Branch-local buffers, rolling summaries, and scoped retrieval provide a practical way to keep unrelated topics from contaminating one another while preserving access to earlier context.

6.1 Future Directions

Adaptive Memory Management. Learn optimal buffer sizes and eviction policies per topic through reinforcement learning.

Automatic Subchat Suggestion. Use topic-shift classifiers to detect context drift and suggest subchat creation proactively.

Cross-Node Synthesis. Controlled mechanisms to merge insights from multiple branches while preserving per-branch isolation guarantees.

Multi-Modal Extension. Apply hierarchical structuring to conversations involving images, code, and documents.

Human Studies. Conduct user studies comparing Subchat Trees to linear chat on real-world tasks (research, coding, creative writing) to measure cognitive load and task completion.

Alternative Backends. Evaluate with GPT-4, Claude, and additional open-source models to assess generalizability across LLM families.

Graph Extensions. Allow directed acyclic graph (DAG) structures where nodes have multiple parents for complex information synthesis.

6.2 Broader Impact

Subchat Trees addresses critical challenges in conversational AI:

- **Privacy:** Isolated contexts prevent sensitive information leakage across topics.
- **Accuracy:** Reduced pollution improves reliability in high-stakes domains (medicine, law, finance).
- **Cost:** Token efficiency reduces barriers for resource-constrained users and organizations.
- **Accessibility:** Explicit conversation structure aids users in navigating complex, multi-topic discussions.

We release our implementation, scenario datasets, and evaluation framework as open-source software to accelerate research in structured conversational AI.

Supplementary Information. The complete source code for Subchat Trees is available at our GitHub repository, including the FastAPI backend, Next.js frontend, final VBRI dataset (JSON format), Kaggle evaluation notebook, evaluation scripts, and buffer-wise result bundles across four buffer sizes.

Acknowledgements. We thank the developers of ChromaDB, SentenceTransformers, and the Llama and Qwen model families for providing the foundational tools that enabled this research. We also acknowledge the LMSYS team for releasing the Chat-1M dataset [19], the Groq team for API access, and the Kaggle platform for GPU compute resources.

Declarations

- **Funding:** Not applicable.
- **Conflict of interest:** The authors declare no competing interests.

- **Ethics approval:** Not applicable.
- **Consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** Test scenarios and evaluation logs are available in the supplementary materials. The LMSYS-Chat-1M dataset [19] is publicly available.
- **Code availability:** Source code is available at: https://github.com/moonmehedi/Subchat-Trees-A-Scalable-Architecture-for-Multi-Threaded-Dialogue-and-Context-Isolation-in-LLM
- **Author contribution:** All authors contributed to the design, implementation, and evaluation of the system. All authors reviewed and approved the final manuscript.

Appendix A Source Conversation Details

The five LMSYS-derived source conversations used to construct the final VBRI dataset are briefly described below.

`22807e655dd042348cb0ee4023672e70`**.** Tests persona isolation (old man, newly-wed, physicist, dog), a Twenty Questions game with repeated role-reversal failures, and LLM meta-discussion. The critical test is whether the model can maintain game state when roles are explicitly swapped.

`6c4992f0aed04dd3bf9a4bc225bb4fb0`**.** Tests containment of a failure mode where the model enters a 15+ iteration repetitive loop ("I am able to process and analyze large amounts of text data..."), alongside medical treatments, electroculture, AI consciousness, literature, and pandemic-safety topics.

`8d10c143f8fc4a7599a5a18778fec112`**.** The broadest source conversation, covering DSP/wavelets, C code, medical genetics (DSD, Klinefelter syndrome), Linux audio, statistics, quantum physics, cosmology, music physics, sci-fi (Liu Cixin), space exploration, Belgium geography, HAL-9000 roleplay, and games.

`eaf06f12a1d74e5ca30a7ca94a7c4128`**.** Tests isolation of an argumentative failure mode where the model incorrectly halves 1 tsp vanilla to 1/4 tsp (should be 1/2 tsp) and defends the error across multiple turns, with interleaved jokes, cookie preferences, and an emoji guessing game.

`ec366fd3e4b5482e8acac750f9b3b55b`**.** Includes repeated user-requested context resets across topics spanning Indian history, logic puzzles, LSAT reasoning, Indian legal code, child nutrition, astrology, elections, Bhutan travel, recipes, therapy, and chess.

`merged_vbri_temporal`**.** The final benchmark merges the five source conversations into a single 309-turn conversation using the VBRI algorithm described in Section 3.5.1. It spans 59 distinct topics and contains 91 topic switches, 64 source-conversation switches, and 25 topic returns. This scenario serves as the testbed for both topic-prefix isolation and recall-probe evaluation.

Appendix B Kaggle Notebook Execution Pipeline

The evaluation pipeline is packaged as a self-contained Kaggle notebook (`kaggle_test_runner.ipynb`) with the following cells:

1. **Cell 1:** Install vLLM dependencies (`vllm`, `triton`).
2. **Cell 2:** Import libraries and verify GPU availability.

3. **Cell 3:** Load Kaggle secrets (GitHub token, Groq key, HuggingFace token) and set `LLM_BACKEND=vllm`.
4. **Cell 4:** Load Qwen-2.5 14B AWQ with `vllm.LLM()`, 4-bit quantization, 91% GPU memory utilization, and `max_model_len=5000`.
5. **Cell 5:** Clone the repository (`kaggle-run` branch) and pull scenario files from Git LFS.
6. **Cell 6:** Register the vLLM model with the backend via `VLLMClient.set_model(llm)`.
7. **Cell 7:** Install backend Python requirements.
8. **Cell 8:** Quick integration test (create node, send message, verify response).
9. **Cell 9:** Execute the configured serverless evaluation: validate scenario JSON files, run baseline and system tests for the selected buffer sizes, generate Markdown tables, raw JSON metrics, recall-probe logs, and buffer-specific component logs.

The notebook version used for the final benchmark targets `merged_realistic_interleaved.json`; the current result section reports the completed 7B, 14B, and 32B runs for buffer sizes $C \in \{5, 10, 20, 40\}$.

Appendix C Per-Topic Confusion Matrix Methodology

Rather than computing a single global confusion matrix, we compute per-topic matrices. For each of the 59 topics τ_i, we count:

- TP_i: Responses correctly scored as belonging to τ_i.
- FP_i: Responses incorrectly attributed to τ_i when the ground truth is a different topic.
- FN_i: Responses where τ_i was expected but a different topic was returned.

The weighted-average metrics reported in the model-specific context-isolation tables (Tables 3, 7, and 11) weight each topic's precision, recall, and F1 by its support (number of ground-truth instances), ensuring that topics with more test steps contribute proportionally to the aggregate. Macro-average metrics treat all topics equally regardless of support.

Appendix D Complete Merged Result Tables

This appendix reproduces the complete merged tables generated from the current result folders for 7B, 14B, and 32B models across buffer sizes $C \in \{5, 10, 20, 40\}$.

D.1 7B Merged Tables

Table D1: 7B: merged context-isolation metrics.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	44.7%	84.0%	71.5%	88.7%	34.9%	80.7%	21.5%	76.2%
Recall	21.1%	58.1%	39.6%	46.1%	15.6%	38.3%	7.8%	33.4%
F1	24.3%	62.9%	44.7%	55.2%	18.7%	45.6%	9.8%	39.3%
Accuracy	21.4%	58.3%	39.8%	46.3%	15.9%	38.5%	8.1%	33.7%
Pollution Rate	78.6%	41.7%	60.2%	53.7%	84.1%	61.5%	91.9%	66.3%
Macro Precision	48.5%	74.4%	70.9%	85.6%	41.7%	76.0%	29.0%	72.8%
Macro Recall	33.7%	59.6%	49.4%	59.7%	24.7%	50.5%	17.8%	41.4%
Macro F1	35.2%	60.8%	53.6%	65.3%	27.2%	55.6%	19.0%	46.5%

Table D2: 7B: merged aggregate recall scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.2448	0.4240	0.2369	0.4094	0.1811	0.4056	0.1557	0.3545
Avg ROUGE-L (F1)	0.1270	0.3004	0.1349	0.3097	0.1016	0.3194	0.0861	0.2574
Avg BLEU-2	0.0364	0.1581	0.0324	0.1259	0.0205	0.1325	0.0172	0.1142
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	158,043	121,521	252,519	188,722	336,942	284,817	472,634	403,852
Avg Tokens per Probe	3,675	2,826	5,873	4,389	7,836	6,624	10,991	9,392
Total Probe Latency	918.00s	710.28s	814.85s	776.91s	767.61s	792.74s	723.47s	703.08s
Avg Probe Latency	21.35s	16.52s	18.95s	18.07s	17.85s	18.44s	16.82s	16.35s

Table D3: 7B: per-topic ROUGE-1 F1 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.2021	0.4254	0.2410	0.2101	0.1660	0.2949	0.1821	0.2635
ai_consciousness	0.2580	0.4540	0.1501	0.2671	0.1476	0.2366	0.1662	0.3648
ai_consciousness_friendship	0.3911	0.4905	0.2495	0.4639	0.2624	0.4216	0.3581	0.5567
ai_meta	0.2663	0.2581	0.2994	0.3636	0.1513	0.2757	0.1769	0.3214
bhutan_travel	0.2285	0.4050	0.1547	0.4433	0.3182	0.0975	0.3304	0.5440
chess	0.3337	0.3140	0.1864	0.5036	0.2256	0.3418	0.2354	0.4324
child_nutrition	0.1623	0.3047	0.2113	0.2341	0.1610	0.2655	0.1934	0.1596
cookies_recipe_halving	0.3874	0.8199	0.4431	0.5997	0.2359	0.6293	0.0841	0.6293
cookies_recipe_halving_argument	0.4291	0.5270	0.3927	0.4427	0.2213	0.6188	0.2921	0.6237
cookies_recipe_halving_corrections	0.3542	0.5931	0.3301	0.6345	0.0661	0.6736	0.1684	0.6911
cookies_recipe_halving_math_errors	0.3121	0.8059	0.4259	0.6610	0.0911	0.6458	0.2279	0.6458
covid_safety	0.3567	0.3862	0.2496	0.3871	0.2354	0.0880	0.1893	0.0698
covid_safety_hiv_aids	0.1604	0.3547	0.2304	0.1854	0.1710	0.3229	0.0599	0.1566
dsp_wavelets	0.1245	0.1577	0.1007	0.1582	0.1588	0.1950	0.0609	0.0963
electroculture	0.0892	0.2333	0.1120	0.2422	0.0149	0.3496	0.0591	0.3274
emoji_game	0.1515	0.1117	0.1811	0.2156	0.2063	0.3143	0.0419	0.3000
game_degree_guess	0.1891	0.4330	0.2290	0.5918	0.2200	0.6593	0.1073	0.4763
game_twenty_questions	0.1354	0.1918	0.2139	0.2957	0.1749	0.4551	0.0499	0.3575
geography_belgium	0.3109	0.6798	0.3982	0.5222	0.4461	0.4358	0.1328	0.6110
humanity_future_150y	0.3412	0.4418	0.2256	0.4732	0.1368	0.4710	0.0458	0.4304
indian_astrology	0.1591	0.3579	0.1761	0.2527	0.0289	0.2203	0.0424	0.0185
indian_history	0.1420	0.6154	0.1211	0.5679	0.1869	0.6870	0.1618	0.5185
indian_legal	0.2342	0.6822	0.2560	0.4519	0.2108	0.3882	0.2389	0.3860
indian_legal_family	0.1410	0.3434	0.2133	0.4432	0.2343	0.6025	0.1953	0.0270
jokes	0.3048	0.6551	0.1566	0.6627	0.1901	0.5973	0.2804	0.6638
karnataka_elections	0.2515	0.3884	0.3563	0.7264	0.0278	0.3956	0.2916	0.2995
linux_audio	0.3791	0.4970	0.3067	0.5253	0.2950	0.4704	0.3300	0.0591
literature_camus	0.2144	0.3049	0.1960	0.2961	0.1765	0.3049	0.2126	0.0270
llm_knowledge	0.3093	0.2710	0.2396	0.3205	0.1585	0.2670	0.0194	0.2085
logic_puzzle	0.2775	0.7113	0.0833	0.6097	0.2430	0.4903	0.1870	0.6263
lsat_medical_conference	0.1429	0.7286	0.1687	0.4985	0.0938	0.6478	0.1206	0.6977
lsat_product_codes	0.1284	0.2628	0.1806	0.2555	0.0268	0.1056	0.0762	0.2708
medical_dsd	0.2218	0.6041	0.1549	0.3869	0.2271	0.3125	0.1974	0.3406
personas_roleplay	0.2727	0.4322	0.2299	0.4697	0.2337	0.7983	0.0653	0.4448
physics_cosmology	0.3249	0.6305	0.3117	0.4609	0.2785	0.0990	0.2125	0.4503
physics_violin_nanoscale	0.3377	0.1142	0.3416	0.3005	0.2473	0.4119	0.1557	0.2977
scifi_films	0.2909	0.2210	0.2248	0.2548	0.0148	0.1974	0.0601	0.2759
scifi_films_hal9000	0.1155	0.3492	0.2967	0.3358	0.2331	0.4793	0.0791	0.4857
scifi_films_hal9000_games	0.1543	0.3055	0.2009	0.2618	0.1588	0.3373	0.0462	0.2645
scifi_films_liu_cixin	0.2654	0.2099	0.2943	0.3290	0.3026	0.4444	0.0996	0.3695
therapy	0.2608	0.3848	0.2003	0.4561	0.3206	0.5734	0.1086	0.0428
therapy_manipulation	0.2090	0.3110	0.1898	0.4475	0.0466	0.4167	0.0984	0.0867
wildfires_alberta	0.2067	0.4619	0.2640	0.3957	0.0396	0.4011	0.2525	0.3232

Table D4: 7B: per-topic ROUGE-L F1 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.1276	0.2810	0.1480	0.0922	0.0779	0.1909	0.0802	0.1579
ai_consciousness	0.1159	0.1982	0.0746	0.1786	0.0779	0.1840	0.0803	0.3455
ai_consciousness_friendship	0.1508	0.2503	0.1083	0.3138	0.1239	0.3253	0.1266	0.3264
ai_meta	0.1241	0.1263	0.1465	0.2466	0.0798	0.1892	0.0750	0.1923
bhutan_travel	0.1183	0.3870	0.0889	0.3488	0.3094	0.0796	0.1550	0.4456
chess	0.1423	0.1433	0.0993	0.4331	0.0986	0.2852	0.1111	0.3430
child_nutrition	0.0724	0.1439	0.0999	0.1448	0.0744	0.2157	0.0889	0.1138
cookies_recipe_halving	0.1635	0.7291	0.3096	0.4749	0.1303	0.4829	0.0664	0.4829
cookies_recipe_halving_argument	0.2162	0.3556	0.2024	0.2710	0.1270	0.4094	0.1445	0.3041
cookies_recipe_halving_corrections	0.1644	0.3690	0.2300	0.4655	0.0555	0.3473	0.1006	0.3740
cookies_recipe_halving_math_errors	0.1545	0.6430	0.2630	0.5428	0.0771	0.5432	0.1348	0.5432
covid_safety	0.1783	0.1501	0.1110	0.2979	0.1050	0.0773	0.0930	0.0640
covid_safety_hiv_aids	0.0702	0.2993	0.1130	0.1440	0.0895	0.3091	0.0434	0.1398
dsp_wavelets	0.0728	0.0791	0.0597	0.0910	0.0812	0.1269	0.0359	0.0713
electroculture	0.0635	0.2119	0.0744	0.2382	0.0100	0.3391	0.0456	0.3265
emoji_game	0.1038	0.0806	0.1274	0.1540	0.1375	0.2738	0.0389	0.1529
game_degree_guess	0.1006	0.2108	0.1279	0.4014	0.1087	0.5149	0.0683	0.3312
game_twenty_questions	0.0771	0.1120	0.1262	0.1632	0.0885	0.3913	0.0369	0.3238
geography_belgium	0.1329	0.5853	0.1663	0.4182	0.2032	0.4071	0.0945	0.3808
humanity_future_150y	0.2547	0.3886	0.1213	0.4630	0.0815	0.4684	0.0316	0.4094
indian_astrology	0.0829	0.2640	0.0999	0.2097	0.0246	0.1735	0.0306	0.0139
indian_history	0.0966	0.4497	0.0895	0.3580	0.1121	0.4609	0.1225	0.4233
indian_legal	0.1155	0.6173	0.1351	0.4182	0.1197	0.3173	0.1194	0.2202
indian_legal_family	0.0832	0.2144	0.1343	0.3735	0.1179	0.5665	0.1316	0.0180
jokes	0.1628	0.5226	0.1325	0.6209	0.1281	0.4887	0.1771	0.5319
karnataka_elections	0.1304	0.2810	0.1987	0.6430	0.0199	0.2706	0.1434	0.2047
linux_audio	0.1954	0.3550	0.1488	0.2373	0.1605	0.2391	0.1449	0.0432
literature_camus	0.1067	0.2587	0.0993	0.2663	0.0858	0.2018	0.0937	0.0240
llm_knowledge	0.1494	0.1492	0.1125	0.1700	0.0784	0.2137	0.0141	0.1513
logic_puzzle	0.1659	0.5798	0.0702	0.5306	0.1371	0.3666	0.1220	0.4811
lsat_medical_conference	0.0772	0.5771	0.1139	0.4758	0.0704	0.6337	0.0778	0.6487
lsat_product_codes	0.0840	0.2416	0.1213	0.2247	0.0255	0.1035	0.0557	0.2610
medical_dsd	0.0988	0.3196	0.0896	0.2373	0.0939	0.2199	0.1138	0.1339
personas_roleplay	0.1096	0.1648	0.1236	0.3371	0.1116	0.7344	0.0435	0.2517
physics_cosmology	0.1861	0.4972	0.2315	0.2868	0.2089	0.0921	0.1324	0.3874
physics_violin_nanoscale	0.1876	0.0913	0.2067	0.1897	0.1649	0.2348	0.1107	0.2558
scifi_films	0.1211	0.1099	0.1172	0.1471	0.0121	0.1268	0.0384	0.2287
scifi_films_hal9000	0.0878	0.1637	0.1540	0.1810	0.1084	0.4247	0.0494	0.2491
scifi_films_hal9000_games	0.0874	0.1468	0.1112	0.1527	0.0907	0.2099	0.0361	0.1391
scifi_films_liu_cixin	0.1503	0.1125	0.1625	0.2857	0.1655	0.3670	0.0630	0.2516
therapy	0.1365	0.3266	0.1202	0.3894	0.1283	0.4230	0.0658	0.0293
therapy_manipulation	0.1061	0.2736	0.1005	0.3151	0.0361	0.3913	0.0679	0.0559
wildfires_alberta	0.1348	0.4571	0.1305	0.3828	0.0321	0.3140	0.0957	0.2352

Table D5: 7B: per-topic BLEU-2 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.0168	0.0916	0.0131	0.0100	0.0034	0.0071	0.0087	0.0126
ai_consciousness	0.0119	0.1148	0.0001	0.0053	0.0010	0.0023	0.0016	0.0352
ai_consciousness_friendship	0.0633	0.1564	0.0126	0.1082	0.0331	0.0756	0.0600	0.1982
ai_meta	0.0260	0.0137	0.0201	0.0341	0.0020	0.0072	0.0031	0.0148
bhutan_travel	0.0156	0.0300	0.0030	0.0650	0.0164	0.0000	0.0804	0.1759
chess	0.0647	0.0375	0.0044	0.2899	0.0125	0.0463	0.0255	0.1829
child_nutrition	0.0038	0.0940	0.0112	0.0058	0.0015	0.0057	0.0082	0.0000
cookies_recipe_halving	0.1040	0.6555	0.1880	0.3241	0.0241	0.3456	0.0000	0.3456
cookies_recipe_halving_argument	0.1375	0.2428	0.1045	0.1009	0.0232	0.3317	0.0649	0.3063
cookies_recipe_halving_corrections	0.0756	0.2668	0.0508	0.3583	0.0000	0.4033	0.0030	0.4252
cookies_recipe_halving_math_errors	0.0444	0.6123	0.1414	0.4137	0.0000	0.3792	0.0074	0.3792
covid_safety	0.0946	0.0588	0.0088	0.0487	0.0100	0.0000	0.0006	0.0000
covid_safety_hiv_aids	0.0076	0.0533	0.0166	0.0015	0.0027	0.0246	0.0000	0.0000
dsp_wavelets	0.0000	0.0000	0.0000	0.0000	0.0001	0.0003	0.0000	0.0000
electroculture	0.0002	0.0019	0.0001	0.0024	0.0000	0.0268	0.0000	0.0187
emoji_game	0.0023	0.0001	0.0057	0.0155	0.0140	0.0386	0.0000	0.0394
game_degree_guess	0.0048	0.1302	0.0337	0.3565	0.0510	0.3866	0.0001	0.1302
game_twenty_questions	0.0003	0.0025	0.0060	0.0107	0.0006	0.1385	0.0000	0.0558
geography_belgium	0.0573	0.3591	0.0846	0.1555	0.1489	0.1770	0.0000	0.3593
humanity_future_150y	0.0324	0.0825	0.0117	0.1073	0.0001	0.1089	0.0000	0.0846
indian_astrology	0.0016	0.1511	0.0021	0.0043	0.0000	0.0011	0.0000	0.0000
indian_history	0.0320	0.3668	0.0041	0.3288	0.0358	0.4292	0.0238	0.3107
indian_legal	0.0279	0.4208	0.0366	0.0891	0.0341	0.0443	0.0337	0.0452
indian_legal_family	0.0017	0.0289	0.0178	0.0827	0.0231	0.2784	0.0592	0.0000
jokes	0.0869	0.4192	0.0154	0.4894	0.0371	0.3817	0.1276	0.5105
karnataka_elections	0.0356	0.1337	0.0713	0.5329	0.0000	0.0543	0.0659	0.0100
linux_audio	0.0843	0.1389	0.0395	0.2167	0.0599	0.1208	0.0518	0.0000
literature_camus	0.0068	0.0100	0.0026	0.0088	0.0033	0.0155	0.0069	0.0000
llm_knowledge	0.0532	0.0126	0.0095	0.0167	0.0011	0.0045	0.0000	0.0008
logic_puzzle	0.0304	0.4504	0.0145	0.2920	0.0229	0.1455	0.0353	0.3373
lsat_medical_conference	0.0211	0.4980	0.0504	0.1472	0.0068	0.3204	0.0111	0.3805
lsat_product_codes	0.0019	0.0061	0.0164	0.0035	0.0000	0.0000	0.0000	0.0060
medical_dsd	0.0055	0.2674	0.0032	0.0567	0.0043	0.0153	0.0046	0.0222
personas_roleplay	0.0444	0.1509	0.0421	0.1342	0.0360	0.6437	0.0000	0.1763
physics_cosmology	0.1455	0.3288	0.0892	0.1347	0.0774	0.0000	0.0257	0.0943
physics_violin_nanoscale	0.0881	0.0007	0.1040	0.0966	0.0830	0.1179	0.0016	0.0180
scifi_films	0.0328	0.0065	0.0102	0.0030	0.0000	0.0003	0.0000	0.0058
scifi_films_hal9000	0.0120	0.0741	0.0420	0.0844	0.0210	0.1106	0.0000	0.1530
scifi_films_hal9000_games	0.0110	0.0420	0.0109	0.0337	0.0062	0.0116	0.0000	0.0074
scifi_films_liu_cixin	0.0249	0.0205	0.0420	0.0241	0.0368	0.1520	0.0000	0.0496
therapy	0.0232	0.1376	0.0168	0.0857	0.0470	0.2326	0.0000	0.0000
therapy_manipulation	0.0086	0.0146	0.0078	0.0849	0.0000	0.0529	0.0000	0.0000
wildfires_alberta	0.0249	0.1150	0.0287	0.0488	0.0000	0.0594	0.0273	0.0186

Table D6: 7B: merged normal conversation performance metrics.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	2,270	2,205	3,384	3,556	5,386	5,685	8,107	8,487
Avg Output Tokens	205	164	214	235	235	241	220	230
Avg Total Tokens	2,476	2,369	3,598	3,791	5,622	5,927	8,327	8,717
Total Input Tokens	701,539	681,497	1,045,593	1,098,821	1,664,366	1,756,760	2,505,151	2,622,497
Total Output Tokens	63,478	50,650	66,087	72,552	72,716	74,536	67,935	70,984
Total Tokens	765,017	732,147	1,111,680	1,171,373	1,737,082	1,831,296	2,573,086	2,693,481
Tokens Per Correct Answer	11,591	4,067	9,038	8,191	35,451	15,389	102,923	25,899
Avg Latency	12.19s	10.14s	10.62s	10.55s	11.70s	11.22s	11.75s	11.71s
Total Latency	3765.47s	3133.41s	3280.66s	3258.46s	3615.37s	3468.37s	3629.73s	3618.90s
Cost per Query	$0.000130	$0.000123	$0.000186	$0.000197	$0.000288	$0.000304	$0.000423	$0.000443
Cost per 1M Queries	$130	$123	$186	$197	$288	$304	$423	$443

Table D7: 7B: merged performance metrics including recall/summarization probes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	158,043	121,521	252,519	188,722	336,942	284,817	472,634	403,852
Avg Tokens per Summarization Probe	3,675	2,826	5,873	4,389	7,836	6,624	10,991	9,392
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	923,060	853,668	1,364,199	1,360,095	2,074,024	2,116,113	3,045,720	3,097,333
Combined Avg Tokens per Call	2,622	2,425	3,876	3,864	5,892	6,012	8,653	8,799

Table D8: 7B: merged normal-turn RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Total Turns	309	309	309	309	309	309	309	309
RAG-Eligible Turns	309	309	309	309	306	309	302	309
RAG Triggered	47	48	42	49	27	12	6	9
Buffer Sufficient	262	261	267	260	279	297	296	300
Retrieval Rate	15.2%	15.5%	13.6%	15.9%	8.8%	3.9%	2.0%	2.9%
Buffer Rate	84.8%	84.5%	86.4%	84.1%	91.2%	96.1%	98.0%	97.1%
Errors	0	0	0	0	3	0	7	0
RAG Disabled	0	0	0	0	0	0	0	0
No Vector Index	0	0	0	0	0	0	0	0

Table D9: 7B: merged recall-probe RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	43
Probe RAG Triggered	1	17	2	9	2	4	0	1
Probe Buffer Sufficient	42	26	41	34	41	39	43	42
Probe Retrieval Rate	2.3%	39.5%	4.7%	20.9%	4.7%	9.3%	0.0%	2.3%
Probe Buffer Rate	97.7%	60.5%	95.3%	79.1%	95.3%	90.7%	100.0%	97.7%
Probe Errors	0	0	0	0	0	0	0	0
Probe RAG Disabled	0	0	0	0	0	0	0	0

D.2 14B Merged Tables

Table D10: 14B: merged context-isolation metrics.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	64.5%	77.8%	81.2%	93.2%	85.0%	89.2%	81.6%	85.3%
Recall	52.6%	74.4%	59.7%	80.2%	60.1%	81.8%	55.8%	69.5%
F1	51.2%	73.9%	64.3%	83.4%	62.5%	83.1%	59.8%	73.1%
Accuracy	52.8%	74.4%	59.9%	80.3%	60.2%	81.9%	56.0%	69.6%
Pollution Rate	47.2%	25.6%	40.1%	19.7%	39.8%	18.1%	44.0%	30.4%
Macro Precision	52.0%	67.5%	75.7%	88.1%	77.6%	85.4%	73.4%	79.5%
Macro Recall	52.7%	68.5%	67.3%	78.7%	65.4%	79.5%	61.8%	65.3%
Macro F1	48.3%	65.8%	67.1%	80.1%	65.1%	79.7%	61.4%	67.9%

Table D11: 14B: merged aggregate recall scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.3428	0.3896	0.3378	0.4201	0.2476	0.4187	0.1745	0.3725
Avg ROUGE-L (F1)	0.2179	0.2634	0.1982	0.3211	0.1320	0.3143	0.0997	0.2648
Avg BLEU-2	0.0967	0.1099	0.0870	0.1338	0.0516	0.1237	0.0182	0.1315
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	136,045	104,023	274,209	156,136	322,902	269,171	359,355	304,196
Avg Tokens per Probe	3,164	2,419	6,377	3,631	7,509	6,260	8,357	7,074
Total Probe Latency	1155.49s	989.76s	1441.58s	997.15s	1284.58s	1196.07s	943.70s	957.29s
Avg Probe Latency	26.87s	23.02s	33.53s	23.19s	29.87s	27.82s	21.95s	22.26s

Table D12: 14B: per-topic ROUGE-1 F1 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.2148	0.3284	0.2399	0.3234	0.2600	0.3084	0.2542	0.4631
ai_consciousness	0.3278	0.3130	0.2393	0.3708	0.3301	0.3407	0.2873	0.4522
ai_consciousness_friendship	0.4958	0.4367	0.3854	0.4820	0.4914	0.3919	0.4393	0.6308
ai_meta	0.3203	0.2989	0.2553	0.2653	0.2773	0.2542	0.3800	0.2896
bhutan_travel	0.2604	0.5093	0.3620	0.3455	0.2670	0.3578	0.0536	0.3580
chess	0.3145	0.3860	0.2136	0.3079	0.3286	0.3164	0.1699	0.0541
child_nutrition	0.0597	0.2338	0.0645	0.2653	0.1847	0.1591	0.0636	0.2512
cookies_recipe_halving	0.3681	0.2827	0.4463	0.8304	0.4433	0.5869	0.1142	0.5869
cookies_recipe_halving_argument	0.5306	0.4528	0.5782	0.4190	0.5882	0.3776	0.2657	0.4026
cookies_recipe_halving_corrections	0.4375	0.4103	0.5455	0.5422	0.5263	0.5215	0.1986	0.5100
cookies_recipe_halving_math_errors	0.3690	0.4563	0.5022	0.4317	0.0601	0.5876	0.2765	0.5876
covid_safety	0.3472	0.3182	0.3711	0.4786	0.3068	0.3505	0.2076	0.0042
covid_safety_hiv_aids	0.2908	0.2367	0.2669	0.3147	0.1855	0.2898	0.2060	0.2313
dsp_wavelets	0.1456	0.1837	0.1425	0.1574	0.1225	0.1488	0.0733	0.1285
electroculture	0.2797	0.1461	0.3299	0.2496	0.1563	0.3105	0.1112	0.5948
emoji_game	0.2098	0.1628	0.2237	0.2578	0.1969	0.1227	0.1229	0.2469
game_degree_guess	0.1818	0.3028	0.2967	0.7091	0.3461	0.7200	0.1608	0.2519
game_twenty_questions	0.1682	0.0996	0.2496	0.1376	0.1858	0.1557	0.0856	0.2444
geography_belgium	0.6840	0.6197	0.6166	0.1516	0.3885	0.6626	0.2201	0.2827
humanity_future_150y	0.6165	0.4975	0.4989	0.4812	0.0258	0.4957	0.0386	0.4494
indian_astrology	0.2555	0.3868	0.3511	0.2678	0.1743	0.3361	0.1135	0.6588
indian_history	0.3762	0.5625	0.1754	0.6824	0.1889	0.7087	0.1823	0.7525
indian_legal	0.3273	0.4579	0.3074	0.4966	0.2822	0.5000	0.2462	0.3847
indian_legal_family	0.4045	0.4332	0.2607	0.4029	0.0270	0.5334	0.1333	0.0503
jokes	0.0833	0.5138	0.1844	0.5936	0.1852	0.7105	0.2451	0.7034
karnataka_elections	0.4453	0.3189	0.4810	0.4940	0.1914	0.4581	0.1667	0.6508
linux_audio	0.7019	0.5343	0.7342	0.6992	0.4084	0.5210	0.4132	0.5544
literature_camus	0.2219	0.3341	0.2500	0.4397	0.2137	0.3065	0.0444	0.0279
llm_knowledge	0.3154	0.2412	0.3287	0.3081	0.3302	0.3605	0.0665	0.3344
logic_puzzle	0.2078	0.5396	0.3003	0.3687	0.2439	0.4073	0.1748	0.6639
lsat_medical_conference	0.3557	0.5696	0.4831	0.6859	0.0942	0.6933	0.0993	0.4019
lsat_product_codes	0.1998	0.2197	0.1917	0.2729	0.0222	0.2556	0.1023	0.0164
medical_dsd	0.3864	0.3557	0.3853	0.4140	0.0182	0.4112	0.2974	0.0194
personas_roleplay	0.3044	0.3812	0.3127	0.3097	0.2711	0.4118	0.2355	0.4390
physics_cosmology	0.5671	0.4351	0.3138	0.4839	0.0361	0.5460	0.0532	0.4037
physics_violin_nanoscale	0.7542	0.8120	0.3249	0.3952	0.5492	0.3971	0.3101	0.4575
scifi_films	0.2425	0.3483	0.2421	0.3645	0.2878	0.3446	0.1211	0.0518
scifi_films_hal9000	0.3087	0.3922	0.3003	0.4535	0.2899	0.4344	0.0444	0.4400
scifi_films_hal9000_games	0.2065	0.2346	0.2326	0.2776	0.1924	0.2086	0.0856	0.2585
scifi_films_liu_cixin	0.4177	0.4797	0.3933	0.5158	0.3931	0.5862	0.0440	0.1220
therapy	0.3477	0.6481	0.3125	0.6585	0.2588	0.5422	0.1864	0.7239
therapy_manipulation	0.3819	0.3748	0.4803	0.4704	0.0119	0.4891	0.0340	0.5561
wildfires_alberta	0.3077	0.5054	0.3505	0.4897	0.3036	0.3812	0.3733	0.3257

Table D13: 14B: per-topic ROUGE-L F1 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.1074	0.1834	0.1087	0.2196	0.1196	0.1834	0.1134	0.2548
ai_consciousness	0.3026	0.1396	0.1288	0.2334	0.2706	0.2488	0.1205	0.4032
ai_consciousness_friendship	0.2266	0.1606	0.1622	0.3122	0.2275	0.1939	0.2401	0.3135
ai_meta	0.1797	0.1663	0.1259	0.1628	0.1203	0.1292	0.1458	0.1805
bhutan_travel	0.1571	0.3628	0.3182	0.2964	0.1263	0.2836	0.0383	0.3128
chess	0.1331	0.2616	0.1048	0.1851	0.1301	0.2210	0.0949	0.0461
child_nutrition	0.0392	0.0956	0.0409	0.2027	0.0793	0.1155	0.0430	0.2077
cookies_recipe_halving	0.1813	0.1653	0.1980	0.7435	0.1873	0.4820	0.0803	0.4820
cookies_recipe_halving_argument	0.2857	0.3270	0.3050	0.2413	0.3205	0.2587	0.1538	0.2468
cookies_recipe_halving_corrections	0.2500	0.2500	0.3192	0.3675	0.3158	0.3954	0.1206	0.3152
cookies_recipe_halving_math_errors	0.2222	0.2586	0.2496	0.2620	0.0420	0.4068	0.1415	0.4068
covid_safety	0.1856	0.1316	0.2742	0.3512	0.1546	0.2762	0.0908	0.0042
covid_safety_hiv_aids	0.2370	0.1243	0.1054	0.2201	0.0940	0.1870	0.1166	0.1349
dsp_wavelets	0.1249	0.1030	0.0757	0.1204	0.0625	0.1038	0.0520	0.0940
electroculture	0.2510	0.0897	0.2919	0.2017	0.0901	0.2993	0.0748	0.5302
emoji_game	0.1422	0.1074	0.1311	0.1828	0.1205	0.1078	0.1102	0.2147
game_degree_guess	0.0973	0.1655	0.1332	0.5280	0.1336	0.5557	0.0863	0.1679
game_twenty_questions	0.1027	0.0681	0.1342	0.0894	0.1048	0.1123	0.0589	0.1881
geography_belgium	0.6061	0.4361	0.2951	0.0831	0.2500	0.4736	0.1292	0.1538
humanity_future_150y	0.5938	0.4608	0.4883	0.4742	0.0159	0.4769	0.0305	0.4226
indian_astrology	0.1294	0.2221	0.1778	0.2448	0.0890	0.2954	0.0922	0.4461
indian_history	0.2475	0.4625	0.1345	0.5405	0.1556	0.5827	0.1042	0.4746
indian_legal	0.1955	0.3175	0.1292	0.4139	0.1349	0.4018	0.1343	0.2621
indian_legal_family	0.3669	0.3946	0.1326	0.3534	0.0187	0.3727	0.1079	0.0294
jokes	0.0606	0.4266	0.1206	0.5342	0.1222	0.6694	0.1383	0.6667
karnataka_elections	0.3047	0.2054	0.2262	0.3333	0.1222	0.3877	0.1190	0.4190
linux_audio	0.6374	0.4979	0.7167	0.6527	0.2073	0.2093	0.2369	0.2280
literature_camus	0.0989	0.2334	0.1101	0.3838	0.1031	0.2319	0.0317	0.0259
llm_knowledge	0.1228	0.1216	0.1389	0.2165	0.1338	0.3065	0.0463	0.2121
logic_puzzle	0.1299	0.4075	0.1769	0.2723	0.1341	0.2826	0.1100	0.4844
lsat_medical_conference	0.1816	0.5198	0.2928	0.6005	0.0659	0.6392	0.0722	0.3690
lsat_product_codes	0.1199	0.1907	0.1362	0.2356	0.0169	0.2035	0.0671	0.0104
medical_dsd	0.1813	0.1541	0.2873	0.3283	0.0114	0.3270	0.1250	0.0194
personas_roleplay	0.1629	0.2525	0.1406	0.2312	0.1133	0.2762	0.1038	0.3039
physics_cosmology	0.2960	0.3839	0.1854	0.3320	0.0206	0.3658	0.0380	0.3421
physics_violin_nanoscale	0.5767	0.6466	0.1987	0.1949	0.4249	0.2249	0.2119	0.2484
scifi_films	0.1104	0.1826	0.1135	0.2689	0.1200	0.2779	0.0781	0.0345
scifi_films_hal9000	0.1417	0.1905	0.1360	0.3012	0.1318	0.2339	0.0333	0.2050
scifi_films_hal9000_games	0.1196	0.1404	0.1121	0.1506	0.0929	0.1396	0.0638	0.1355
scifi_films_liu_cixin	0.1994	0.3140	0.2050	0.4087	0.2184	0.5081	0.0440	0.0871
therapy	0.1508	0.4341	0.1364	0.5528	0.1280	0.3222	0.1017	0.5654
therapy_manipulation	0.2596	0.1635	0.3704	0.3949	0.0119	0.4457	0.0212	0.4634
wildfires_alberta	0.1506	0.4087	0.1525	0.3827	0.1328	0.3013	0.1659	0.2733

Table D14: 14B: per-topic BLEU-2 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.0055	0.0352	0.0102	0.0287	0.0146	0.0153	0.0184	0.0926
ai_consciousness	0.0256	0.0166	0.0045	0.0333	0.0241	0.0244	0.0131	0.0875
ai_consciousness_friendship	0.1301	0.0978	0.0638	0.1361	0.1404	0.0494	0.0917	0.3078
ai_meta	0.0372	0.0150	0.0134	0.0057	0.0187	0.0049	0.0685	0.0125
bhutan_travel	0.0076	0.1192	0.0283	0.0194	0.0132	0.0234	0.0000	0.0237
chess	0.0280	0.0101	0.0004	0.0133	0.0528	0.0148	0.0008	0.0000
child_nutrition	0.0000	0.0088	0.0000	0.0092	0.0037	0.0000	0.0000	0.0055
cookies_recipe_halving	0.1018	0.0147	0.1781	0.6779	0.1476	0.2447	0.0001	0.2447
cookies_recipe_halving_argument	0.2416	0.1021	0.2869	0.1026	0.3142	0.0434	0.0037	0.0972
cookies_recipe_halving_corrections	0.1573	0.1257	0.2562	0.2130	0.2683	0.1991	0.0035	0.1989
cookies_recipe_halving_math_errors	0.0893	0.1331	0.2590	0.1573	0.0000	0.2211	0.0161	0.2211
covid_safety	0.0422	0.0241	0.0353	0.1091	0.0178	0.0283	0.0018	0.0000
covid_safety_hiv_aids	0.0247	0.0199	0.0379	0.0197	0.0032	0.0222	0.0007	0.0150
dsp_wavelets	0.0000	0.0003	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
electroculture	0.0065	0.0001	0.0182	0.0029	0.0002	0.0138	0.0000	0.2963
emoji_game	0.0153	0.0024	0.0136	0.0112	0.0175	0.0025	0.0016	0.0413
game_degree_guess	0.0025	0.0403	0.0688	0.5187	0.0729	0.4783	0.0013	0.0096
game_twenty_questions	0.0008	0.0000	0.0201	0.0000	0.0021	0.0003	0.0000	0.0042
geography_belgium	0.4252	0.3883	0.3481	0.0005	0.2031	0.3440	0.0046	0.1007
humanity_future_150y	0.2943	0.1373	0.1447	0.1311	0.0000	0.1458	0.0000	0.0976
indian_astrology	0.0052	0.0488	0.0325	0.0047	0.0017	0.0230	0.0031	0.3342
indian_history	0.1571	0.3048	0.0199	0.5262	0.0123	0.4971	0.0173	0.6198
indian_legal	0.0681	0.0985	0.0552	0.1386	0.0453	0.1413	0.0506	0.0437
indian_legal_family	0.0578	0.0886	0.0298	0.0545	0.0000	0.1683	0.0069	0.0000
jokes	0.0005	0.3785	0.0050	0.3503	0.0308	0.5097	0.0361	0.5080
karnataka_elections	0.1348	0.0372	0.2206	0.1681	0.0076	0.1458	0.0061	0.3900
linux_audio	0.4441	0.1668	0.4696	0.4476	0.1207	0.1922	0.1654	0.2537
literature_camus	0.0075	0.0205	0.0130	0.0879	0.0103	0.0127	0.0000	0.0000
llm_knowledge	0.0237	0.0043	0.0460	0.0132	0.0388	0.0337	0.0000	0.0341
logic_puzzle	0.0163	0.2133	0.0471	0.0371	0.0596	0.0719	0.0040	0.3404
lsat_medical_conference	0.0730	0.2202	0.1947	0.4022	0.0001	0.3661	0.0009	0.0542
lsat_product_codes	0.0029	0.0010	0.0013	0.0066	0.0000	0.0056	0.0001	0.0000
medical_dsd	0.0696	0.1232	0.0469	0.0643	0.0000	0.0648	0.0757	0.0000
personas_roleplay	0.0581	0.0466	0.0471	0.0209	0.0274	0.0929	0.0160	0.1076
physics_cosmology	0.3036	0.1249	0.1219	0.1819	0.0000	0.2040	0.0000	0.0571
physics_violin_nanoscale	0.6226	0.6431	0.0999	0.1477	0.2751	0.1499	0.0914	0.1320
scifi_films	0.0142	0.0308	0.0177	0.0374	0.0183	0.0215	0.0001	0.0000
scifi_films_hal9000	0.1116	0.1118	0.0844	0.1199	0.0593	0.1071	0.0000	0.1765
scifi_films_hal9000_games	0.0155	0.0026	0.0238	0.0123	0.0067	0.0004	0.0000	0.0058
scifi_films_liu_cixin	0.1551	0.2025	0.1438	0.2035	0.1278	0.2541	0.0000	0.0000
therapy	0.0728	0.3720	0.0605	0.3061	0.0207	0.2017	0.0052	0.5191
therapy_manipulation	0.0472	0.0375	0.1180	0.0954	0.0000	0.1376	0.0000	0.2028
wildfires_alberta	0.0591	0.1593	0.0564	0.1378	0.0427	0.0407	0.0783	0.0182

Table D15: 14B: merged normal conversation performance metrics.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	1,818	1,919	3,152	3,092	4,659	5,396	5,685	5,981
Avg Output Tokens	133	145	170	182	177	215	148	160
Avg Total Tokens	1,952	2,064	3,321	3,274	4,837	5,611	5,833	6,141
Total Input Tokens	561,795	592,924	973,916	955,556	1,439,776	1,667,280	1,756,657	1,848,024
Total Output Tokens	41,238	44,890	52,376	56,130	54,831	66,582	45,676	49,555
Total Tokens	603,033	637,814	1,026,292	1,011,686	1,494,607	1,733,862	1,802,333	1,897,579
Tokens Per Correct Answer	3,700	2,773	5,548	4,079	8,036	6,853	10,418	8,826
Avg Latency	16.71s	16.16s	15.00s	13.58s	16.06s	17.69s	14.58s	14.64s
Total Latency	5161.96s	4993.71s	4635.76s	4195.00s	4962.52s	5466.95s	4504.21s	4524.19s
Cost per Query	$0.000102	$0.000108	$0.000171	$0.000169	$0.000247	$0.000287	$0.000296	$0.000312
Cost per 1M Queries	$102	$108	$171	$169	$247	$287	$296	$312

Table D16: 14B: merged performance metrics including recall/summarization probes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	136,045	104,023	274,209	156,136	322,902	269,171	359,355	304,196
Avg Tokens per Summarization Probe	3,164	2,419	6,377	3,631	7,509	6,260	8,357	7,074
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	739,078	741,837	1,300,501	1,167,822	1,817,509	2,003,033	2,161,688	2,201,775
Combined Avg Tokens per Call	2,100	2,107	3,695	3,318	5,163	5,690	6,141	6,255

Table D17: 14B: merged normal-turn RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Total Turns	309	309	309	309	309	309	309	309
RAG-Eligible Turns	309	309	309	309	306	309	284	291
RAG Triggered	36	29	26	29	27	30	22	21
Buffer Sufficient	273	280	283	280	279	279	262	270
Retrieval Rate	11.7%	9.4%	8.4%	9.4%	8.8%	9.7%	7.7%	7.2%
Buffer Rate	88.3%	90.6%	91.6%	90.6%	91.2%	90.3%	92.3%	92.8%
Errors	0	0	0	0	3	0	25	18
RAG Disabled	0	0	0	0	0	0	0	0
No Vector Index	0	0	0	0	0	0	0	0

Table D18: 14B: merged recall-probe RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	41
Probe RAG Triggered	25	10	19	1	16	2	7	0
Probe Buffer Sufficient	18	33	24	42	27	41	36	41
Probe Retrieval Rate	58.1%	23.3%	44.2%	2.3%	37.2%	4.7%	16.3%	0.0%
Probe Buffer Rate	41.9%	76.7%	55.8%	97.7%	62.8%	95.3%	83.7%	100.0%
Probe Errors	0	0	0	0	0	0	0	2
Probe RAG Disabled	0	0	0	0	0	0	0	0

D.3 32B Merged Tables

Table D19: 32B: merged context-isolation metrics.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Precision	88.0%	88.5%	87.3%	94.4%	89.9%	93.4%	84.9%	90.0%
Recall	64.6%	83.1%	69.2%	80.2%	67.2%	83.4%	63.6%	74.4%
F1	68.6%	83.2%	71.1%	83.7%	69.6%	85.2%	65.7%	78.2%
Accuracy	64.7%	83.2%	69.3%	80.3%	67.3%	83.5%	63.8%	74.4%
Pollution Rate	35.3%	16.8%	30.7%	19.7%	32.7%	16.5%	36.2%	25.6%
Macro Precision	83.6%	85.1%	81.4%	91.8%	86.7%	93.0%	78.3%	88.2%
Macro Recall	73.9%	81.9%	75.2%	79.8%	75.8%	86.8%	71.6%	74.0%
Macro F1	74.4%	80.8%	73.9%	81.9%	74.5%	87.1%	68.4%	76.7%

Table D20: 32B: merged aggregate recall scores and probe cost.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Avg ROUGE-1 (F1)	0.2687	0.4328	0.3674	0.4206	0.3338	0.4283	0.1503	0.3918
Avg ROUGE-L (F1)	0.1767	0.2830	0.2274	0.2982	0.2009	0.3016	0.0878	0.2624
Avg BLEU-2	0.0502	0.1452	0.1023	0.1253	0.0731	0.1247	0.0234	0.1206
Topics Probed	43	43	43	43	43	43	43	43
Total Probe Tokens	119,832	131,487	249,908	175,875	373,285	264,906	324,171	310,617
Avg Tokens per Probe	2,787	3,058	5,812	4,090	8,681	6,161	7,539	7,224
Total Probe Latency	1959.92s	2495.09s	2965.46s	2722.82s	3002.40s	2796.51s	1888.54s	2288.25s
Avg Probe Latency	45.58s	58.03s	68.96s	63.32s	69.82s	65.04s	43.92s	53.22s

Table D21: 32B: per-topic ROUGE-1 F1 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.3593	0.2817	0.3653	0.4000	0.2252	0.3455	0.1798	0.3985
ai_consciousness	0.2589	0.2945	0.2690	0.3389	0.2391	0.2936	0.3008	0.3789
ai_consciousness_friendship	0.3647	0.4213	0.4780	0.3983	0.3721	0.4308	0.4514	0.5257
ai_meta	0.3122	0.2893	0.2543	0.3714	0.3302	0.3252	0.2917	0.3237
bhutan_travel	0.0681	0.4394	0.3073	0.3465	0.2359	0.3370	0.2375	0.3178
chess	0.1337	0.2837	0.2213	0.3636	0.3053	0.3150	0.2119	0.3994
child_nutrition	0.0638	0.4402	0.2080	0.3421	0.2978	0.2587	0.0932	0.1436
cookies_recipe_halving	0.2179	0.2867	0.4158	0.4301	0.3452	0.5273	0.2761	0.4919
cookies_recipe_halving_argument	0.4920	0.5110	0.5490	0.5207	0.5615	0.5908	0.0746	0.5483
cookies_recipe_halving_corrections	0.3374	0.4641	0.4379	0.5014	0.4885	0.5215	0.4712	0.6383
cookies_recipe_halving_math_errors	0.2980	0.3601	0.4378	0.4779	0.3725	0.6658	0.0625	0.6711
covid_safety	0.0666	0.3727	0.2711	0.3954	0.4353	0.3837	0.0105	0.0278
covid_safety_hiv_aids	0.2625	0.2839	0.3945	0.2798	0.2540	0.3451	0.0425	0.1702
dsp_wavelets	0.0219	0.1659	0.1858	0.1408	0.0889	0.1393	0.1619	0.1412
electroculture	0.3497	0.3807	0.3422	0.3136	0.2393	0.2780	0.0274	0.4615
emoji_game	0.0530	0.2049	0.0714	0.1611	0.2295	0.2025	0.1935	0.3711
game_degree_guess	0.2363	0.5035	0.4760	0.5303	0.2430	0.5520	0.2832	0.4530
game_twenty_questions	0.0622	0.3232	0.2625	0.2580	0.1781	0.2983	0.0848	0.2668
geography_belgium	0.2431	0.7078	0.7831	0.7617	0.7337	0.6773	0.0315	0.4696
humanity_future_150y	0.0726	0.4762	0.4314	0.5175	0.4013	0.5255	0.1046	0.5148
indian_astrology	0.2117	0.3719	0.2398	0.4101	0.3303	0.2766	0.0158	0.3860
indian_history	0.4359	0.5957	0.4598	0.6909	0.4674	0.7200	0.0348	0.6828
indian_legal	0.2776	0.5057	0.3402	0.5200	0.2718	0.4604	0.3042	0.4163
indian_legal_family	0.3731	0.4562	0.3778	0.3916	0.4031	0.3633	0.0952	0.0862
jokes	0.1695	0.5631	0.3401	0.5741	0.2282	0.6022	0.2323	0.6625
karnataka_elections	0.5378	0.4565	0.6090	0.5541	0.6195	0.5926	0.0519	0.5620
linux_audio	0.3931	0.6580	0.2950	0.7880	0.4305	0.6247	0.4551	0.5430
literature_camus	0.4306	0.4123	0.2305	0.3801	0.2489	0.3654	0.2259	0.1739
llm_knowledge	0.3466	0.2486	0.3029	0.2781	0.3465	0.3035	0.0167	0.2689
logic_puzzle	0.1705	0.6264	0.3270	0.4738	0.3732	0.5411	0.2595	0.5215
lsat_medical_conference	0.3634	0.5178	0.4884	0.5458	0.5304	0.5755	0.0389	0.5687
lsat_product_codes	0.1460	0.1701	0.3030	0.2417	0.1482	0.2594	0.0091	0.0149
medical_dsd	0.5132	0.6422	0.3523	0.4580	0.3178	0.4941	0.1636	0.4150
personas_roleplay	0.1545	0.3648	0.3643	0.3044	0.2461	0.3614	0.0453	0.3707
physics_cosmology	0.1074	0.7212	0.6006	0.5472	0.3183	0.5584	0.3599	0.2720
physics_violin_nanoscale	0.1750	0.6464	0.6553	0.2805	0.3333	0.4191	0.0645	0.3710
scifi_films	0.1024	0.3740	0.3040	0.3259	0.2366	0.2978	0.0216	0.3197
scifi_films_hal9000	0.2781	0.4240	0.3800	0.3768	0.3339	0.4161	0.2787	0.5191
scifi_films_hal9000_games	0.2012	0.2663	0.2249	0.2076	0.2246	0.3794	0.0868	0.3407
scifi_films_liu_cixin	0.4142	0.5891	0.4684	0.6078	0.3903	0.5851	0.0361	0.5776
therapy	0.4405	0.4694	0.2214	0.5247	0.1860	0.4114	0.0187	0.1756
therapy_manipulation	0.3303	0.5221	0.2782	0.3647	0.2313	0.3978	0.0424	0.5239
wildfires_alberta	0.7071	0.5173	0.4730	0.3929	0.5624	0.3991	0.0171	0.3627

Table D22: 32B: per-topic ROUGE-L F1 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.1629	0.1596	0.2037	0.2397	0.1026	0.2435	0.0986	0.2798
ai_consciousness	0.1431	0.1848	0.1368	0.1885	0.1195	0.1808	0.2061	0.2197
ai_consciousness_friendship	0.1563	0.2272	0.2537	0.2318	0.1697	0.2893	0.2069	0.3204
ai_meta	0.1306	0.1736	0.1518	0.2337	0.1336	0.2127	0.1329	0.1871
bhutan_travel	0.0481	0.3653	0.2703	0.3242	0.1454	0.2806	0.1625	0.2877
chess	0.0840	0.1576	0.1277	0.1935	0.1371	0.1752	0.0984	0.1895
child_nutrition	0.0435	0.2912	0.1350	0.2606	0.2286	0.2040	0.0529	0.0764
cookies_recipe_halving	0.1401	0.1470	0.2013	0.2826	0.1548	0.3682	0.1448	0.3573
cookies_recipe_halving_argument	0.2812	0.2912	0.2876	0.2781	0.3102	0.3415	0.0672	0.3133
cookies_recipe_halving_corrections	0.1963	0.2762	0.2438	0.3215	0.3155	0.3110	0.2670	0.4113
cookies_recipe_halving_math_errors	0.1806	0.1918	0.2242	0.2726	0.2118	0.4465	0.0573	0.4430
covid_safety	0.0378	0.1660	0.1295	0.2747	0.2902	0.2531	0.0105	0.0232
covid_safety_hiv_aids	0.2179	0.1986	0.3423	0.1840	0.2021	0.2619	0.0243	0.0947
dsp_wavelets	0.0170	0.1068	0.1577	0.1021	0.0650	0.0898	0.0813	0.1045
electroculture	0.3174	0.3150	0.3105	0.2616	0.1439	0.2691	0.0229	0.2418
emoji_game	0.0419	0.1449	0.0602	0.1316	0.1538	0.1805	0.1505	0.3286
game_degree_guess	0.1519	0.2517	0.2579	0.3509	0.1229	0.2724	0.1358	0.3328
game_twenty_questions	0.0470	0.2216	0.1702	0.2095	0.1205	0.2496	0.0633	0.1783
geography_belgium	0.1389	0.5994	0.7367	0.7224	0.6586	0.5550	0.0210	0.3000
humanity_future_150y	0.0495	0.3692	0.3327	0.5030	0.1692	0.4986	0.0624	0.4598
indian_astrology	0.1696	0.2049	0.1358	0.2275	0.1619	0.2618	0.0095	0.3465
indian_history	0.3333	0.4574	0.3218	0.5164	0.3370	0.5000	0.0348	0.3724
indian_legal	0.2310	0.4211	0.2019	0.4145	0.1332	0.3915	0.1458	0.2881
indian_legal_family	0.2533	0.4253	0.3082	0.2819	0.3260	0.2110	0.0714	0.0682
jokes	0.1186	0.4725	0.1814	0.5237	0.1456	0.4696	0.1333	0.4984
karnataka_elections	0.3761	0.3130	0.3851	0.4326	0.4040	0.3951	0.0461	0.4147
linux_audio	0.1792	0.5152	0.1475	0.5671	0.2007	0.3317	0.1993	0.2921
literature_camus	0.3375	0.1921	0.1099	0.2255	0.1121	0.2248	0.1146	0.1234
llm_knowledge	0.2470	0.2336	0.1464	0.2258	0.2333	0.2312	0.0167	0.1974
logic_puzzle	0.1172	0.4451	0.1761	0.2939	0.1964	0.3539	0.1628	0.2893
lsat_medical_conference	0.2782	0.4139	0.3159	0.4230	0.3749	0.4929	0.0278	0.5375
lsat_product_codes	0.0902	0.1438	0.2100	0.1869	0.1159	0.2340	0.0079	0.0081
medical_dsd	0.3383	0.3028	0.1869	0.2563	0.1944	0.3222	0.1273	0.2916
personas_roleplay	0.0935	0.2240	0.1783	0.1948	0.1157	0.2290	0.0302	0.2356
physics_cosmology	0.0785	0.3806	0.3571	0.4306	0.2200	0.3206	0.2271	0.2184
physics_violin_nanoscale	0.1125	0.3802	0.3925	0.1789	0.2143	0.2058	0.0323	0.2039
scifi_films	0.0599	0.2044	0.1761	0.1673	0.1205	0.2285	0.0108	0.2211
scifi_films_hal9000	0.1716	0.2026	0.1700	0.1988	0.1948	0.2484	0.1434	0.2977
scifi_films_hal9000_games	0.1096	0.1341	0.1065	0.1213	0.1184	0.2638	0.0615	0.2044
scifi_films_liu_cixin	0.2973	0.3492	0.2928	0.5120	0.1963	0.4946	0.0301	0.3448
therapy	0.3688	0.2985	0.1195	0.3692	0.1023	0.2547	0.0187	0.1311
therapy_manipulation	0.1920	0.3417	0.1429	0.2199	0.1215	0.3115	0.0424	0.2374
wildfires_alberta	0.4591	0.2747	0.2809	0.2881	0.3455	0.3097	0.0146	0.3116

Table D23: 32B: per-topic BLEU-2 across buffers.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
ai_chat_tools	0.0435	0.0100	0.0397	0.0510	0.0072	0.0158	0.0010	0.0507
ai_consciousness	0.0076	0.0153	0.0128	0.0232	0.0048	0.0109	0.0140	0.0447
ai_consciousness_friendship	0.0723	0.0778	0.1927	0.0575	0.0414	0.0744	0.1491	0.1902
ai_meta	0.0359	0.0140	0.0067	0.0461	0.0367	0.0202	0.0171	0.0212
bhutan_travel	0.0000	0.0490	0.0091	0.0160	0.0051	0.0268	0.0194	0.0137
chess	0.0003	0.0124	0.0080	0.0537	0.0524	0.0120	0.0072	0.0664
child_nutrition	0.0000	0.1020	0.0014	0.0235	0.0095	0.0029	0.0000	0.0001
cookies_recipe_halving	0.0086	0.0246	0.1124	0.1197	0.0427	0.2121	0.0225	0.1917
cookies_recipe_halving_argument	0.1584	0.1943	0.2612	0.1855	0.2123	0.2724	0.0000	0.2352
cookies_recipe_halving_corrections	0.0532	0.1553	0.1810	0.2390	0.1654	0.2065	0.2178	0.3684
cookies_recipe_halving_math_errors	0.0368	0.0641	0.1382	0.1478	0.0492	0.3809	0.0000	0.3860
covid_safety	0.0000	0.0334	0.0061	0.0462	0.0853	0.0390	0.0000	0.0000
covid_safety_hiv_aids	0.0045	0.0292	0.0562	0.0253	0.0081	0.0390	0.0000	0.0039
dsp_wavelets	0.0000	0.0001	0.0001	0.0000	0.0000	0.0000	0.0002	0.0000
electroculture	0.0230	0.0398	0.0221	0.0135	0.0013	0.0065	0.0000	0.0969
emoji_game	0.0000	0.0047	0.0001	0.0004	0.0109	0.0008	0.0210	0.0531
game_degree_guess	0.0021	0.2231	0.2233	0.2389	0.0162	0.2464	0.0235	0.1251
game_twenty_questions	0.0000	0.0378	0.0190	0.0117	0.0037	0.0253	0.0000	0.0155
geography_belgium	0.0092	0.5862	0.6837	0.5830	0.5051	0.3415	0.0000	0.2922
humanity_future_150y	0.0000	0.1126	0.0686	0.1525	0.0525	0.1549	0.0000	0.1271
indian_astrology	0.0014	0.0445	0.0024	0.0537	0.0339	0.0073	0.0000	0.0615
indian_history	0.2017	0.3420	0.2062	0.4688	0.2199	0.4842	0.0000	0.4438
indian_legal	0.0164	0.1489	0.0720	0.1419	0.0268	0.0997	0.0632	0.0603
indian_legal_family	0.0383	0.1027	0.0411	0.0444	0.0560	0.0367	0.0010	0.0000
jokes	0.0036	0.3549	0.1232	0.4182	0.0608	0.3134	0.0369	0.3528
karnataka_elections	0.2233	0.1528	0.3479	0.2507	0.3306	0.2920	0.0000	0.2861
linux_audio	0.0654	0.4264	0.0308	0.5697	0.1055	0.2957	0.1563	0.2753
literature_camus	0.0827	0.0621	0.0056	0.0497	0.0112	0.0404	0.0110	0.0002
llm_knowledge	0.0315	0.0035	0.0227	0.0069	0.0312	0.0103	0.0000	0.0052
logic_puzzle	0.0113	0.2971	0.0820	0.1127	0.1006	0.2282	0.0554	0.1900
lsat_medical_conference	0.0791	0.1617	0.2212	0.1903	0.1897	0.2320	0.0000	0.2213
lsat_product_codes	0.0008	0.0003	0.0158	0.0026	0.0000	0.0061	0.0000	0.0000
medical_dsd	0.1895	0.3432	0.0281	0.0931	0.0141	0.1551	0.0066	0.0716
personas_roleplay	0.0017	0.0723	0.0940	0.0297	0.0117	0.0661	0.0000	0.0639
physics_cosmology	0.0016	0.5549	0.3539	0.2033	0.0968	0.2486	0.1576	0.0117
physics_violin_nanoscale	0.0024	0.4295	0.3353	0.0398	0.1217	0.1732	0.0000	0.1243
scifi_films	0.0000	0.0470	0.0258	0.0211	0.0129	0.0110	0.0000	0.0162
scifi_films_hal9000	0.0298	0.1229	0.0670	0.1095	0.0852	0.1141	0.0251	0.2024
scifi_films_hal9000_games	0.0025	0.0124	0.0042	0.0027	0.0070	0.0341	0.0000	0.0264
scifi_films_liu_cixin	0.0652	0.3296	0.1404	0.2925	0.0797	0.2703	0.0000	0.3287
therapy	0.1285	0.1151	0.0090	0.1663	0.0078	0.0683	0.0000	0.0004
therapy_manipulation	0.0197	0.1637	0.0062	0.0406	0.0076	0.0456	0.0000	0.1329
wildfires_alberta	0.5053	0.1687	0.1193	0.0444	0.2224	0.0433	0.0000	0.0291

Table D24: 32B: merged normal conversation performance metrics.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Normal Conversation Turns	309	309	309	309	309	309	309	309
Avg Input Tokens	2,038	2,119	3,165	3,156	5,043	5,292	6,252	6,405
Avg Output Tokens	139	158	173	176	191	219	162	170
Avg Total Tokens	2,177	2,277	3,338	3,331	5,234	5,511	6,413	6,575
Total Input Tokens	629,813	654,649	977,889	975,125	1,558,268	1,635,311	1,931,805	1,979,202
Total Output Tokens	42,810	48,954	53,478	54,260	59,026	67,566	49,943	52,427
Total Tokens	672,623	703,603	1,031,367	1,029,385	1,617,294	1,702,877	1,981,748	2,031,629
Tokens Per Correct Answer	3,363	2,738	4,819	4,151	7,775	6,600	10,060	8,833
Avg Latency	36.91s	35.86s	37.52s	33.71s	38.99s	39.29s	34.77s	33.56s
Total Latency	11404.56s	11081.32s	11594.14s	10417.47s	12046.50s	12139.55s	10745.07s	10370.19s
Cost per Query	$0.000113	$0.000119	$0.000172	$0.000172	$0.000267	$0.000282	$0.000326	$0.000334
Cost per 1M Queries	$113	$119	$172	$172	$267	$282	$326	$334

Table D25: 32B: merged performance metrics including recall/summarization probes.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Summarization Probe Calls	43	43	43	43	43	43	43	43
Summarization Probe Tokens	119,832	131,487	249,908	175,875	373,285	264,906	324,171	310,617
Avg Tokens per Summarization Probe	2,787	3,058	5,812	4,090	8,681	6,161	7,539	7,224
Combined Evaluated Calls	352	352	352	352	352	352	352	352
Combined Total Tokens	792,455	835,090	1,281,275	1,205,260	1,990,579	1,967,783	2,305,919	2,342,246
Combined Avg Tokens per Call	2,251	2,372	3,640	3,424	5,655	5,590	6,551	6,654

Table D26: 32B: merged normal-turn RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Total Turns	309	309	309	309	309	309	309	309
RAG-Eligible Turns	309	309	309	309	309	309	300	306
RAG Triggered	61	80	84	66	71	67	64	48
Buffer Sufficient	248	229	225	243	238	242	236	258
Retrieval Rate	19.7%	25.9%	27.2%	21.4%	23.0%	21.7%	21.3%	15.7%
Buffer Rate	80.3%	74.1%	72.8%	78.6%	77.0%	78.3%	78.7%	84.3%
Errors	0	0	0	0	0	0	9	3
RAG Disabled	0	0	0	0	0	0	0	0
No Vector Index	0	0	0	0	0	0	0	0

Table D27: 32B: merged recall-probe RAG decision breakdown.

Metric	Buffer 5		Buffer 10		Buffer 20		Buffer 40	
	Base	System	Base	System	Base	System	Base	System
Probe Turns	43	43	43	43	43	43	43	43
RAG-Eligible Probe Turns	43	43	43	43	43	43	43	42
Probe RAG Triggered	23	38	40	23	26	8	31	1
Probe Buffer Sufficient	20	5	3	20	17	35	12	41
Probe Retrieval Rate	53.5%	88.4%	93.0%	53.5%	60.5%	18.6%	72.1%	2.4%
Probe Buffer Rate	46.5%	11.6%	7.0%	46.5%	39.5%	81.4%	27.9%	97.6%
Probe Errors	0	0	0	0	0	0	0	1
Probe RAG Disabled	0	0	0	0	0	0	0	0

References

[1] Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. arXiv preprint arXiv:2307.03172 (2023)

[2] Laban, P., Schnabel, T., Bennett, P., Hearst, M.A.: LLMs get lost in multi-turn conversation. arXiv preprint arXiv:2404.07961 (2024)

[3] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903 (2022)

[4] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171 (2023)

[5] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601 (2023)

[6] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Gianinazzi, L., Gajda, J., Lehmann, T., Podstawski, M., Niewiadomski, H., Nyczyk, P., Hoefler, T.: Graph of thoughts: Solving elaborate problems with large language models. arXiv preprint arXiv:2308.09687 (2023)

[7] Elsner, M., Charniak, E.: You talking to me? a corpus and algorithm for conversation disentanglement. In: Proceedings of ACL-08: HLT, pp. 834–842. Association for Computational Linguistics, Columbus, Ohio (2008). https://aclanthology.org/P08-1095/

[8] Jiang, J.-Y., Chen, F., Chen, Y.-Y., Wang, W.: Learning to disentangle interleaved conversational threads with a siamese hierarchical network and similarity ranking. In: Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers), pp. 1812–1822. Association for Computational Linguistics, New Orleans, Louisiana (2018). https://doi.org/10.18653/v1/N18-1164 . https://aclanthology.org/N18-1164/

[9] Kummerfeld, J.K., Gouravajhala, S.R., Peper, J.J., Athreya, V., Gunasekara, C., Ganhotra, J., Patel, S.S., Polymenakos, L., Lasecki, W.: A large-scale corpus for conversation disentanglement. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, pp. 3846–3856 (2019)

[10] Serban, I.V., Sordoni, A., Bengio, Y., Courville, A., Pineau, J.: Building end-to-end dialogue systems using generative models: Advances to the Ubuntu dialogue corpus. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30

(2016)

[11] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S.G., Stoica, I., Gonzalez, J.E.: MemGPT: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560 (2023)

[12] Wu, Q., Yu, Z.: Stateful memory-augmented transformers for efficient dialogue modeling. In: Findings of the Association for Computational Linguistics: EACL 2024, pp. 853–867. Association for Computational Linguistics, St. Julian's, Malta (2024). https://doi.org/10.18653/v1/2024.findings-eacl.57 . https://aclanthology.org/2024.findings-eacl.57/

[13] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474 (2020)

[14] Li, G., Hammoud, H.A.A.K., Itani, H., Khizbullin, D., Ghanem, B.: CAMEL: Communicative agents for "mind" exploration of large language model society. arXiv preprint arXiv:2303.17760 (2023)

[15] Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y.: Evaluating very long-term conversational memory of LLM agents. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 13851–13870. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.acl-long.747 . https://aclanthology.org/2024.acl-long.747/

[16] Xu, J., Szlam, A., Weston, J.: Beyond goldfish memory: Long-term open-domain conversation. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 5180–5197. Association for Computational Linguistics, Dublin, Ireland (2022). https://doi.org/10.18653/v1/2022.acl-long.356 . https://aclanthology.org/2022.acl-long.356/

[17] Xu, X., Gou, Z., Wu, W., Niu, Z.-Y., Wu, H., Wang, H., Wang, S.: Long time no see! open-domain conversation with long-term persona memory. In: Findings of the Association for Computational Linguistics: ACL 2022, pp. 2639–2650. Association for Computational Linguistics, Dublin, Ireland (2022). https://doi.org/10.18653/v1/2022.findings-acl.207 . https://aclanthology.org/2022.findings-acl.207/

[18] Jiang, H., Wu, Q., Luo, X., Li, D., Lin, C.-Y., Yang, Y., Qiu, L.: LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 1658–1677. Association for Computational Linguistics, Bangkok, Thailand (2024). https://doi.org/10.18653/v1/2024.acl-long.91 . https://aclanthology.org/2024.acl-long.91/

[19] Zheng, L., Chiang, W.-L., Sheng, Y., Li, T., Zhuang, S., Wu, Z., Zhuang, Y., Li, Z., Lin, Z., Xing, E.P., Gonzalez, J.E., Stoica, I., Zhang, H.: LMSYS-Chat-1M: A large-scale real-world LLM conversation dataset. arXiv preprint arXiv:2309.11998 (2024)

[20] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)

[21] Sutskever, I., Vinyals, O., Le, Q.V.: Sequence to sequence learning with neural networks. In: Advances in Neural Information Processing Systems 27. Curran Associates, Inc., ??? (2014). https://papers.nips.cc/paper/5346-sequence-to-sequence-learning-with-neural

[22] Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate. arXiv preprint arXiv:1409.0473 (2014) https://doi.org/10.48550/arXiv.1409.0473

[23] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Advances in Neural Information Processing Systems 30. Curran Associates, Inc., ??? (2017). https://proceedings.neurips.cc/paper/7181-attention-is-all-you-need

[24] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems 33. Curran Associates, Inc., ??? (2020). https://papers.nips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html

[25] Wei, J., Bosma, M., Zhao, V.Y., Guu, K., Yu, A.W., Lester, B., Du, N., Dai, A.M., Le, Q.V.: Finetuned language models are zero-shot learners. In: International Conference on Learning Representations (2022). https://openreview.net/forum?id=gEZrGCozdqR

[26] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., Schulman, J., Hilton, J., Kelton, F., Miller, L., Simens, M., Askell, A., Welinder, P., Christiano, P., Leike, J., Lowe, R.: Training language models to follow instructions with human feedback. arXiv preprint arXiv:2203.02155 (2022)

[27] Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on

Natural Language Processing (EMNLP-IJCNLP), pp. 3982–3992. Association for Computational Linguistics, ??? (2019). https://doi.org/10.18653/v1/D19-1410

[28] Chroma: Chroma: The AI-native open-source embedding database. https://www.trychroma.com/. Accessed: 2025 (2024)

[29] Nielsen, J.: Usability engineering. Morgan Kaufmann, San Francisco, CA (1994)

[30] Norman, D.: The Design of Everyday Things: Revised and Expanded Edition. Basic Books, New York, NY (2013)

[31] Lin, C.-Y.: ROUGE: A package for automatic evaluation of summaries. In: Text Summarization Branches Out: Proceedings of the ACL-04 Workshop, pp. 74–81. Association for Computational Linguistics, Barcelona, Spain (2004)

[32] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics, pp. 311–318. Association for Computational Linguistics, Philadelphia, Pennsylvania, USA (2002). https://doi.org/10.3115/1073083.1073135

[33] Team, Q.: Qwen2.5: A party of foundation models. arXiv preprint arXiv:2412.15115 (2025)